

EDUDEMOS



Demostrador 3×1:

Energía Solar, eólica y captación de agua
en un solo proyecto educativo

Financiado por la Unión Europea. Las opiniones y puntos de vista expresados solo comprometen a su(s) autor(es) y no reflejan necesariamente los de la Unión Europea o los de la Agencia Ejecutiva Europea de Educación y Cultura (EACEA). Ni la Unión Europea ni la EACEA pueden ser considerados responsables de ellos.



EduDemoS 3in1 © 2025 by Gerda Stetter Stiftung – Technik macht Spaß!, Sergio Alonso Fundation, FINNOVA REGIO, GBS St. Gallen e IES El Rincón is licensed under

Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International. To view a copy of this license, visit <https://creativecommons.org/licenses/by-nc-sa/4.0/>

1. ENSAMBLAJE DEL DEMOSTRADOR 3x1

El montaje del Demostrador 3x1 se realizará de forma modular, abordando cada sección por separado en los apartados correspondientes de esta guía. Este enfoque busca simplificar el proceso y facilitar su comprensión, permitiendo que el montaje sea claro y accesible para quienes lo lleven a cabo. Cada paso está diseñado con el objetivo de garantizar un resultado funcional y efectivo, incluso para quienes no cuenten con experiencia previa en ensamblaje de demostradores.

Uno de los objetivos principales de este proyecto es fomentar la autonomía de los centros educativos, permitiéndoles crear sus propios demostradores si cuentan con impresoras 3D en sus instalaciones. Por esta razón, hemos puesto a disposición los archivos de impresión necesarios para la fabricación de los componentes, los cuales pueden descargarse a través del siguiente enlace: [Archivos de impresión 3D](#). Con esta herramienta, buscamos empoderar a las instituciones educativas para que integren estos demostradores en sus actividades de forma independiente y eficiente.

1.1. Listado de materiales

- 1 x **ESP32 DEVKIT V1** de 30 pines
- 2 x **Mini breadboard** de 170 puntos
- 1 x **Módulo DHT11**, de ancho máximo 14 mm y puede o no tener led incluido
- 10/12* x **Cables macho-macho** 10 cm
- 5 x **Cables macho-macho** 20 cm
- 3 x **Cables macho-hembra** 10 cm
- 10 x **Cables macho-hembra** 20 cm
- 5 x **Resistencias** de 1k ohmio
- 1 x **Condensador** de 100nF / 0.1uF
- 3 x **Leds** de colores (Rojo, Verde y Azul preferiblemente)
- 1 x **Placa solar** dimensiones máximas: 45 mm x 60 mm
- 1 x **Sensor nivel de agua**, 6 cm x 2 cm
- 1 x **Motor DC** de corriente continua de 5V, dimensiones máximas: 28 mm diámetro y 45 mm largo
- 1 x **Servomotor SG-90**

- 1 x **Detector de tensión**
- 2 x **Fotoresistencias** independientes GL5528
- 1 x **Rodamiento 6203** puede ser del tipo zz o 2rs: 40 mm diámetro exterior, 17 mm diámetro interior y 12 mm de alto
- 3 x **Espiga de madera**: 8 mm de diámetro y 40mm de largo
- 1 x **Engranaje motor**: 2 mm diámetro interior, 10 dientes y 5 mm alto
- 1 x **Cable de datos USB a microUSB**
- 1 x **Cinta adhesiva de aluminio**, 5 cm de ancho
- 1 x **Cinta adhesiva de doble cara**, 3 cm de ancho

* Si el Motor DC no viene con los cables soldados, se necesitarán 12 cables macho-macho de 10 cm.

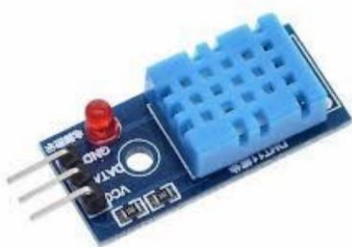
Herramientas que podrías necesitar:

- **Soldador**: si el motor DC o el panel solar no vienen con cables.
- **Destornillador plano de 2 mm**: para el sensor de voltaje.

1. 2. Montaje del Demostrador de Agua + Solar

1. 2. 1. Montaje electrónico (Parte de Agua)

Comenzaremos realizando el montaje del captador de agua. Para realizar este montaje completo necesitarás el siguiente material:



1 x Sensor DHT-11



3 x Resistencias de 1 kΩ

Encargado de medir la temperatura y la humedad ambientales.

Usadas para limitar la corriente que fluye por los diodos LED.



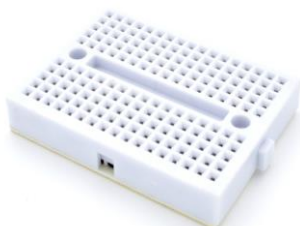
3 x LEDs de Colores (Rojo, Verde y Azul preferiblemente)

Empleados como indicadores visuales del rango de la temperatura medida.



1 x Cable dupont (Macho-Macho) 10cm

Necesarios para realizar las conexiones entre los diferentes componentes en la protoboard.



2 x Mini Breadboard

Componente necesario para conectar la placa microcontroladora con los diferentes componentes



6 x Cable dupont (Macho-Hembra) 20cm

Necesarios para realizar las conexiones entre los diferentes componentes en la protoboard.



1 x Sensor nivel de agua

Empleado para medir el nivel de agua en el tanque.

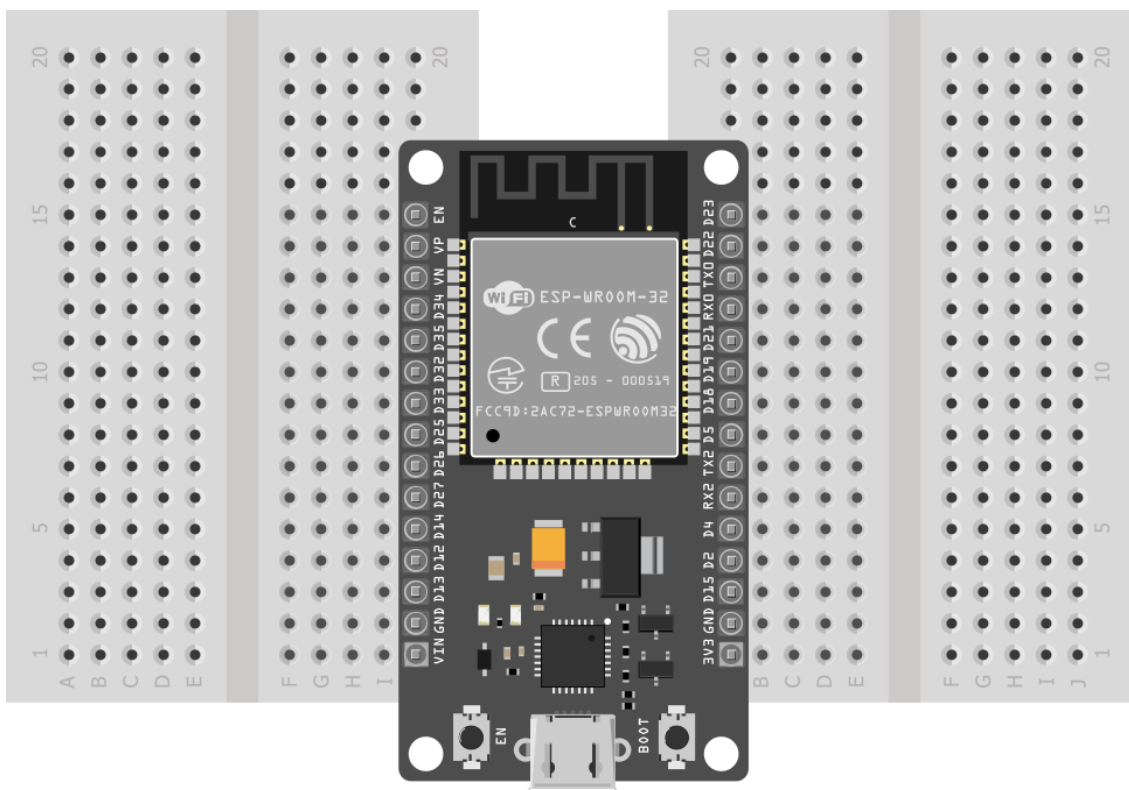


1 x ESP32 DEVKIT V1 (30 pins)

Encargado de ejecutar la programación que controla el funcionamiento de la electrónica.

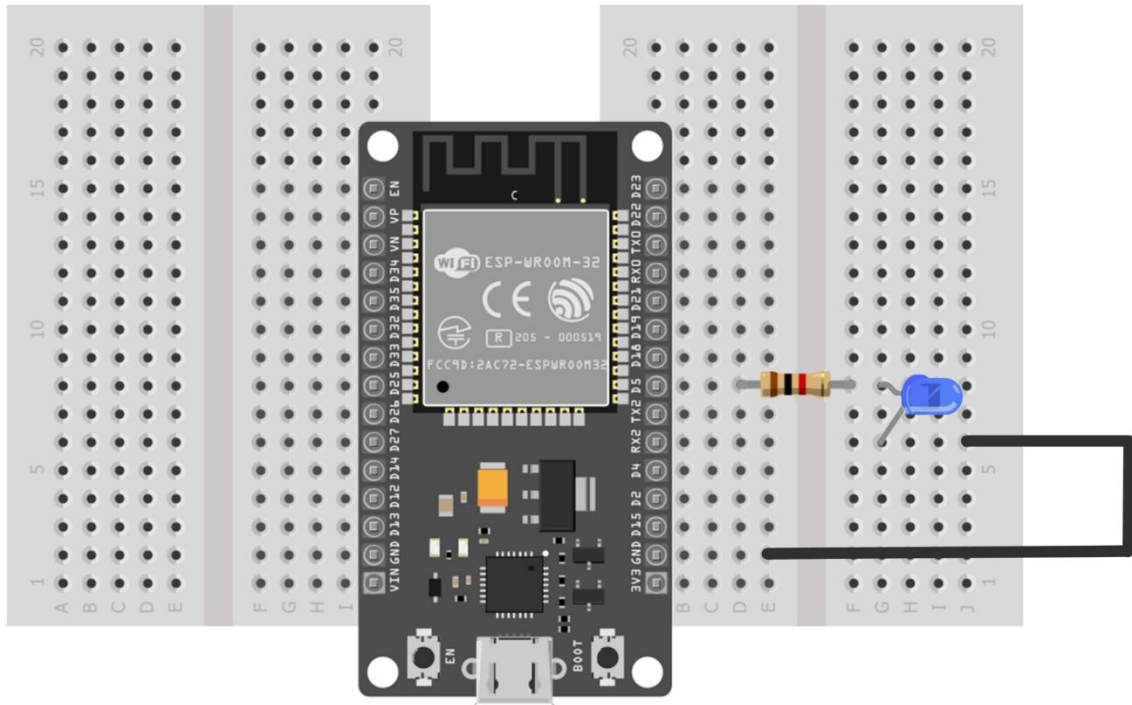
El objetivo del montaje será obtener el valor de la temperatura y la humedad ambientales y, en función del valor de la temperatura, encender un LED u otro como indicadores de peligro por alta o baja temperatura. De esta manera si la temperatura está por debajo de los 20°C se enciende el LED Azul, si se está por encima de los 25°C se enciende el LED Rojo y si está entre ambos valores (entre 20 y 25°C) se enciende el LED Verde.

El primer paso es colocar la placa con ESP32 sobre dos protoboard pequeñas para tener espacio suficiente para luego conectar los componentes que vamos a necesitar. A continuación se puede ver el resultado:



Una vez tengamos el microcontrolador colocado en las dos protoboards, empezaremos a conectar los LEDs de colores con sus resistencias limitadoras. Vamos a empezar colocando el LED Azul y una de las resistencias de 1 k Ω en serie. Conectaremos el positivo del LED en el **pin D5**, y utilizaremos cualquier pin GND para conectar el negativo del circuito. A continuación se puede ver el montaje indicado.

Nota: El diodo LED nos indica la polaridad de la conexión a simple vista con el tamaño de sus patas de conexión. La más larga coincide con el ánodo, o positivo, y la más corta se trata del cátodo, o negativo. En el diagrama, la pata larga es la que va conectada en serie con la resistencia.



Para comprobar el correcto funcionamiento del montaje, podemos cargar el siguiente código en el ESP32:

```
#define ledPin1Cold 5 // Definimos el pin donde está conectado el LED Azul

// Bloque de inicialización
void setup() {

    // Configuramos el modo de funcionamiento del pin del LED como salida
    pinMode(ledPin1Cold, OUTPUT);

}

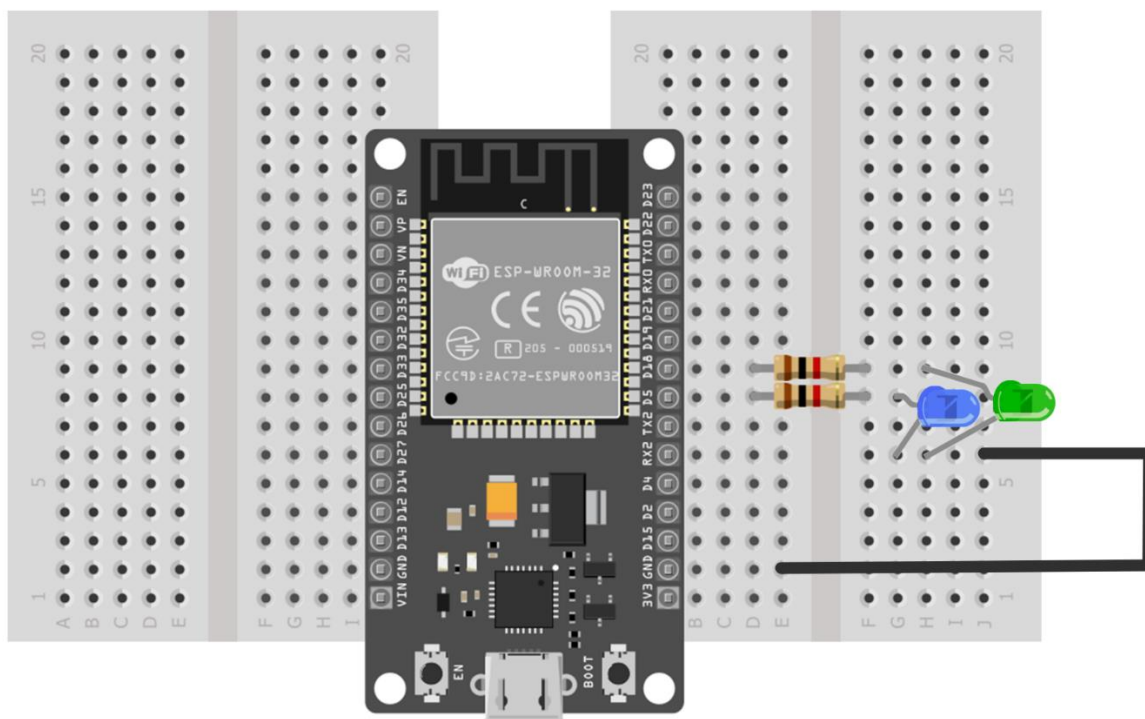
// Bucle principal del código
void loop() {

    digitalWrite(ledPin1Cold, LOW); // Apagamos el LED
    delay(1000); // Esperamos 1 segundo = 1000 milisegundos
    digitalWrite(ledPin1Cold, HIGH); // Encendemos el LED
    delay(1000); // Esperamos 1 segundo
```

}

Este pequeño programa hace parpadear el LED Azul, cambiando su estado entre encendido y apagado cada segundo. Si no funciona correctamente, comprueba que el LED se encuentre bien orientado (recuerda que el terminal más largo es el positivo, mientras el corto es el negativo) y conectado adecuadamente con la resistencia limitadora.

El siguiente paso del montaje será añadir un segundo LED. Esta vez colocaremos y conectaremos el LED de color Verde en el **pin D18** con su resistencia. Puedes ver el esquema de montaje indicado en la siguiente imagen.



Vamos a comprobar de nuevo que el montaje es correcto. Vamos a utilizar ahora el código que encontrarás a continuación para verificar que el nuevo LED también está adecuadamente colocado.

```
#define ledPin1Cold 5 // Definimos el pin donde está conectado el LED Azul
#define ledPin2Good 18 // Definimos el pin donde está conectado el LED Verde

// Bloque de inicialización
void setup() {

    // Configuramos el modo de funcionamiento de los pines de los LED como salida
    pinMode(ledPin1Cold, OUTPUT);
    pinMode(ledPin2Good, OUTPUT);

}

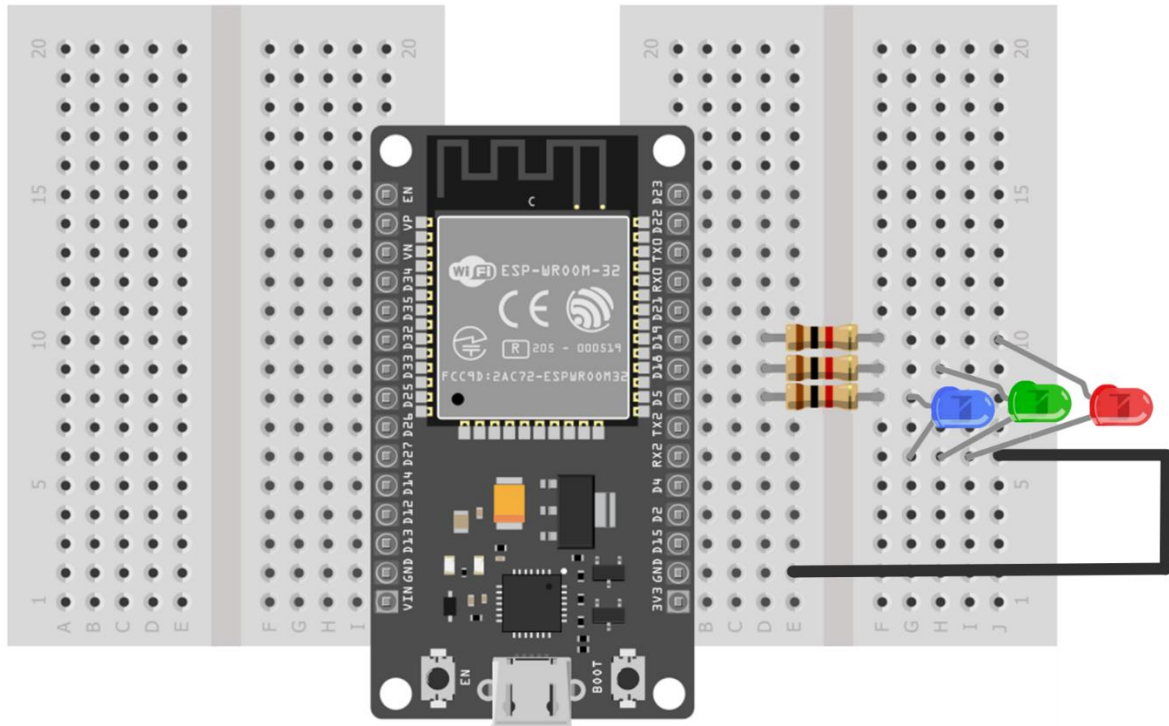
// Bucle principal del código
void loop() {

    digitalWrite(ledPin1Cold, LOW); // Apagamos el LED Azul
    digitalWrite(ledPin2Good, LOW); // Apagamos el LED Verde
    delay(1000); // Esperamos 1 segundo = 1000 milisegundos
    digitalWrite(ledPin1Cold, HIGH); // Encendemos el LED Azul
    digitalWrite(ledPin2Good, HIGH); // Encendemos el LED Verde
    delay(1000); // Esperamos 1 segundo

}
```

Como puedes ver, el objetivo del código es el mismo: encender y apagar, esta vez, ambos LED cambiando de estado cada segundo. Si el LED Azul y el Verde cambian de estado correctamente, podemos dar por válido el montaje actual y continuar con el siguiente paso.

Para terminar de montar los LEDs de colores que necesitamos para cumplir con el objetivo establecido con esta parte del Demostrador, necesitamos conectar el último LED de color Rojo en el **pin D19**. Procede a montar el diodo LED y la última resistencia correspondiente, como se muestra en la siguiente imagen.



Comprobaremos que está correcto realizando la misma programación que en los casos anteriores, generando un parpadeo, ahora, en los tres LEDs de colores. El código para ello es el que sigue.

```
#define ledPin1Cold 5 // Definimos el pin donde está conectado el LED Azul
#define ledPin2Good 18 // Definimos el pin donde está conectado el LED Verde
#define ledPin3Heat 19 // Definimos el pin donde está conectado el LED Rojo

// Bloque de inicialización
void setup() {

    // Configuramos el modo de funcionamiento de los pines de los LED como salida
    pinMode(ledPin1Cold, OUTPUT);
    pinMode(ledPin2Good, OUTPUT);
    pinMode(ledPin3Heat, OUTPUT);

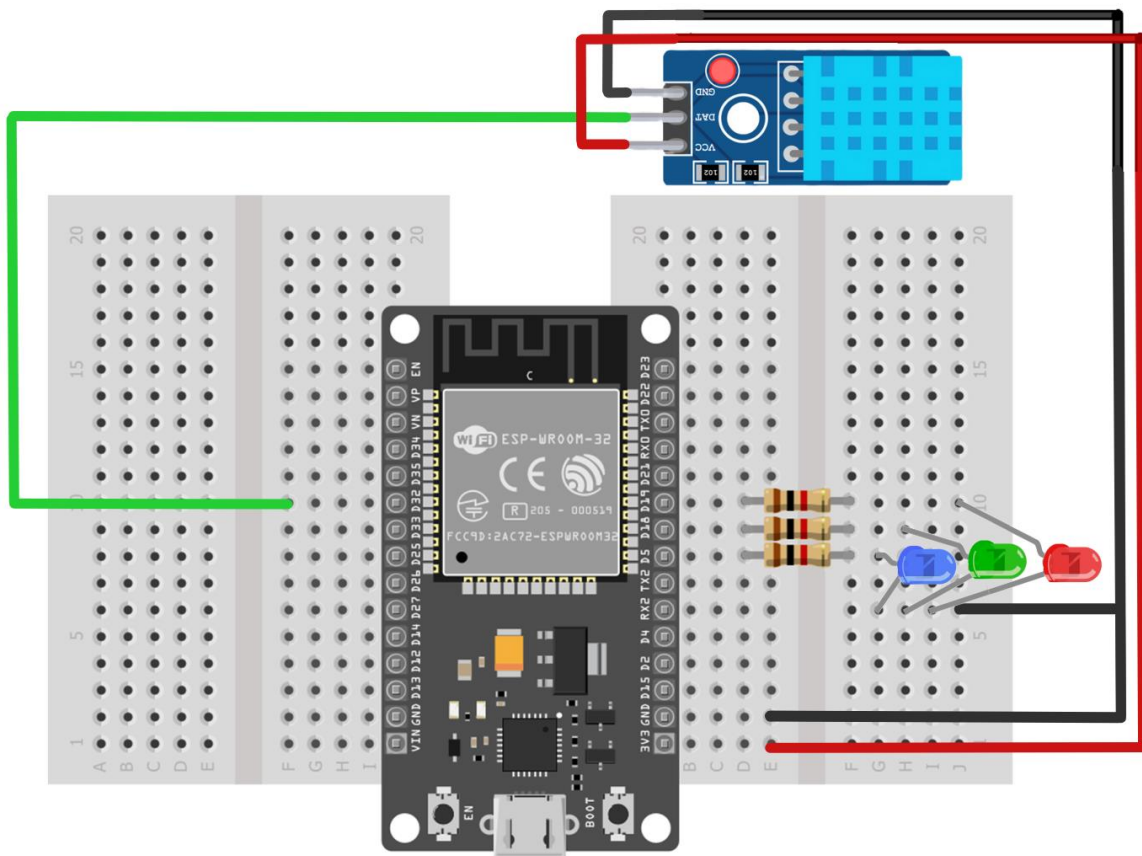
}

// Bucle principal del código
void loop() {
```

```
digitalWrite(ledPin1Cold, LOW); // Apagamos el LED Azul
digitalWrite(ledPin2Good, LOW); // Apagamos el LED Verde
digitalWrite(ledPin3Heat, LOW); // Apagamos el LED Rojo
delay(1000); // Esperamos 1 segundo = 1000 milisegundos
digitalWrite(ledPin1Cold, HIGH); // Encendemos el LED Azul
digitalWrite(ledPin2Good, HIGH); // Encendemos el LED Verde
digitalWrite(ledPin3Heat, HIGH); // Encendemos el LED Rojo
delay(1000); // Esperamos 1 segundo

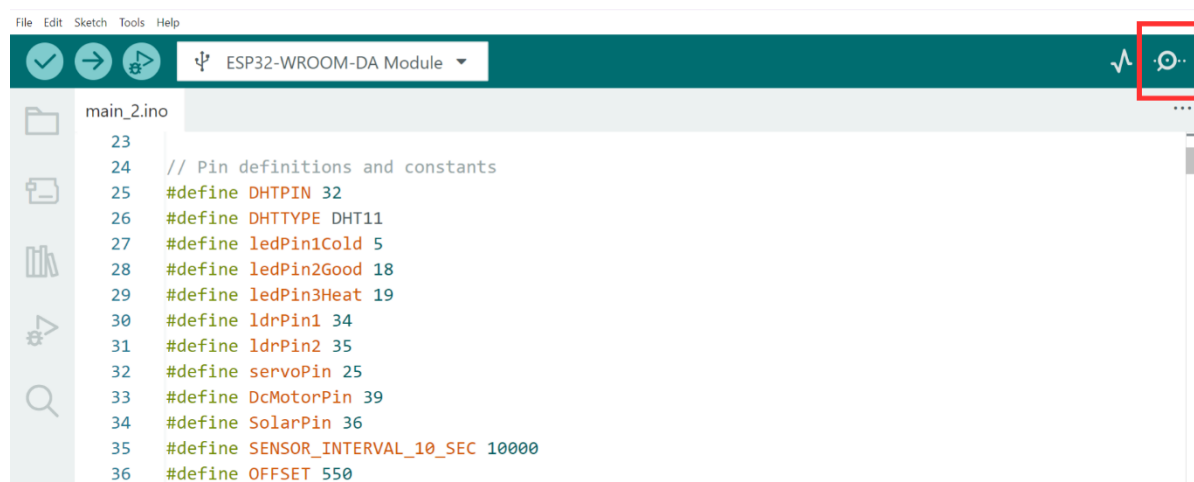
}
```

El próximo paso es agregar y conectar el sensor DHT11 en el **pin D32**, ya sea como un componente individual o en un módulo. A continuación se presenta el montaje realizado hasta este momento junto con las conexiones del sensor DHT11.



Nota: Comprobar que las conexiones GND y VCC del sensor DHT11 se corresponden con la descripción de los pines reales del componente, ya que puede ser diferente a la que aparece en el diagrama.

Pasamos a comprobar el correcto funcionamiento del sensor DHT-11. A continuación, dispones de un código de prueba de todo el montaje realizado para esta parte del montaje. Carga el programa y verás cómo, además de que los LEDs de colores parpadean como han hecho hasta ahora, la placa con ESP32 enviará por el Monitor Serial (puedes ver el botón donde abrirlo en la siguiente imagen) cada par de segundos las medidas obtenidas de temperatura ambiental (en °C) y humedad relativa del aire (en %).



```
#include <DHT.h> // Incluimos la librería necesaria para controlar de DHT

#define ledPin1Cold 5 // Definimos el pin donde está conectado el LED Azul
#define ledPin2Good 18 // Definimos el pin donde está conectado el LED Verde
#define ledPin3Heat 19 // Definimos el pin donde está conectado el LED Rojo
#define DHTPIN 32 // Definimos el pin donde está conectado el DHT
#define DHTTYPE DHT11 // Definimos el tipo de sensor DHT que vamos a utilizar

// Creamos un objeto de tipo DHT con la configuración correspondiente
DHT myDHT11(DHTPIN, DHTTYPE);

// Bloque de inicialización
void setup() {

    // Configuramos el modo de funcionamiento de los pines de los LED como salida
```

```
pinMode(ledPin1Cold, OUTPUT);
pinMode(ledPin2Good, OUTPUT);
pinMode(ledPin3Heat, OUTPUT);

// Inicializamos el monitor Serial
Serial.begin(115200);
// Inicializamos el sensor DHT
myDHT11.begin();

}

// Bucle principal del código
void loop() {

    digitalWrite(ledPin1Cold, LOW); // Apagamos el LED Azul
    digitalWrite(ledPin2Good, LOW); // Apagamos el LED Verde
    digitalWrite(ledPin3Heat, LOW); // Apagamos el LED Rojo
    delay(1000); // Esperamos 1 segundo = 1000 milisegundos
    digitalWrite(ledPin1Cold, HIGH); // Encendemos el LED Azul
    digitalWrite(ledPin2Good, HIGH); // Encendemos el LED Verde
    digitalWrite(ledPin3Heat, HIGH); // Encendemos el LED Rojo
    delay(1000); // Esperamos 1 segundo

    //Obtenemos las medidas del sensor DHT
    float temperatura = myDHT11.readTemperature();
    float humedad = myDHT11.readHumidity();

    // Mostramos los valores por monitor Serial
    Serial.print("Temperatura: ");
    Serial.println(temperatura);
    Serial.print("Humedad: ");
    Serial.println(humedad);

}
```

Con el monitor abierto, comprueba que el desplegable que se ubica más a la derecha sobre el propio Monitor tenga seleccionado el valor “115200”. Ahora podrás ver los datos que envía el ESP32 tomados del DHT-11 de manera constante y verificar que el montaje hasta este punto funciona correctamente.



1. 2. 2. Montaje electrónico (Parte Solar)

Comenzaremos realizando el montaje del componente solar. Para realizar este montaje completo necesitarás el siguiente material:



1 x Servomotor SG-90

Utilizado para mover el Demostrador solar y orientarlo hacia la luz solar.



2 x Fotorresistencias independientes

Empleados para medir el nivel de luminosidad recibida.



2 x Resistencias 1k ohm

Utilizadas solamente junto con las LDR independientes para montar un partidor de tensión controlado por intensidad lumínica.



9 x Cable dupont (Macho-Macho) 10cm

Necesarios para realizar las conexiones entre los diferentes componentes en la protoboard.



5 x Cable dupont (Macho-Macho) 20cm

Necesarios para realizar las conexiones entre los diferentes componentes en la protoboard.



3 x Cable dupont (Macho-Hembra) 10cm

Necesarios para realizar las conexiones entre los diferentes componentes en la protoboard.



4 x Cable dupont (Macho-Hembra) 20cm

Necesarios para realizar las conexiones entre los diferentes componentes en la protoboard.



1 x Detector de tensión

Necesarios para realizar las conexiones entre los diferentes componentes en la protoboard.

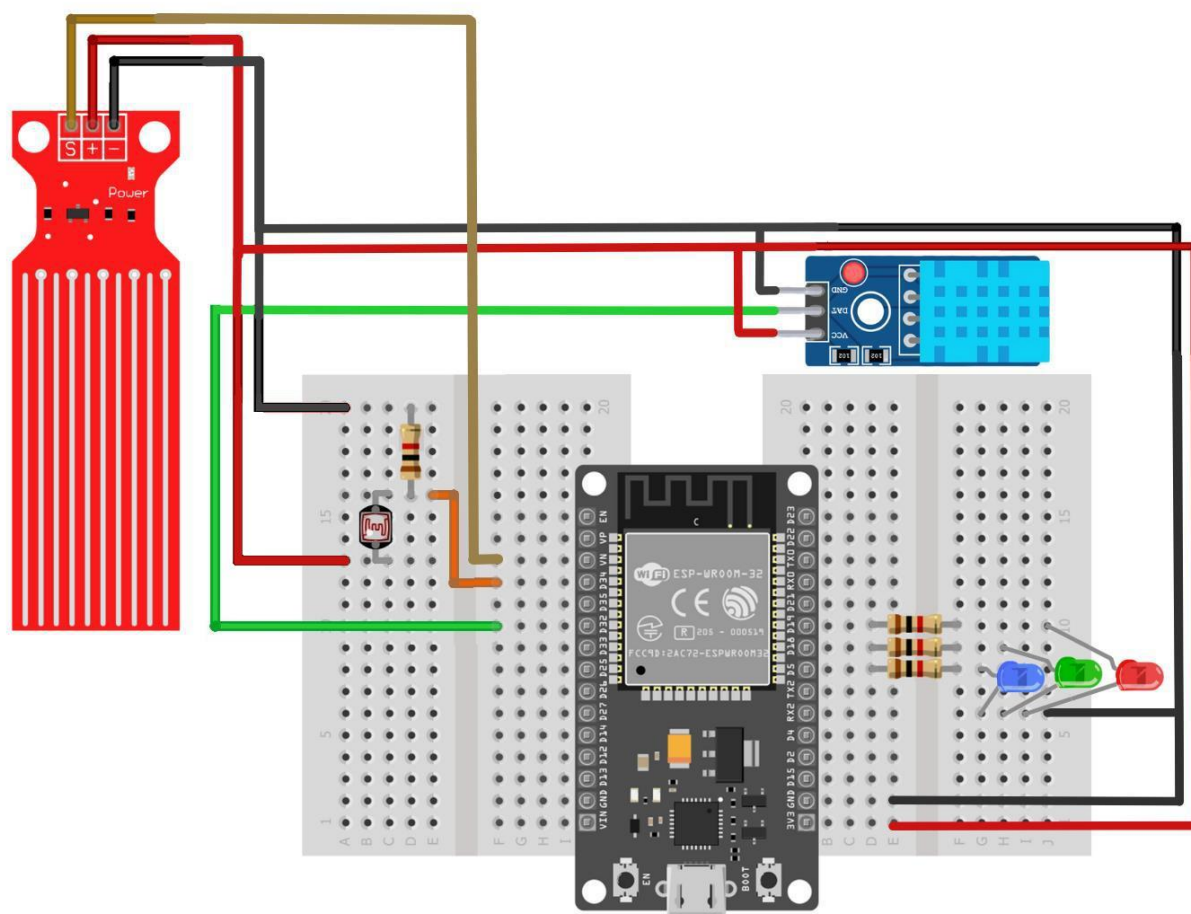


1 x Panel solar

Empleado para detectar luz solar.

El objetivo del montaje será obtener la orientación de la incidencia de la luz solar con las LDR y rotar el Demostrador para que incida la mayor cantidad de rayos solares sobre el panel.

El montaje del componente solar va a comenzar desde el último montaje que se vio en el apartado anterior. En este montaje se añade una LDR y una resistencia en serie, formando un partidor de tensión controlado por la intensidad lumínica. Para obtener el valor del sensor se va a utilizar la salida digital del mismo, que se va a conectar al **pin D34** del ESP32. El montaje se puede ver a continuación:



Para comprobar el funcionamiento de los nuevos elementos añadidos al montaje, vamos a emplear un código que solamente tome la medida de la LDR para encender el LED Rojo con mayor o menor intensidad según la luminosidad medida por la fotorresistencia. Puedes ver el código a cargar a continuación.

```
#define ldrPin1 34 // Definimos el pin donde está conectado la LDR

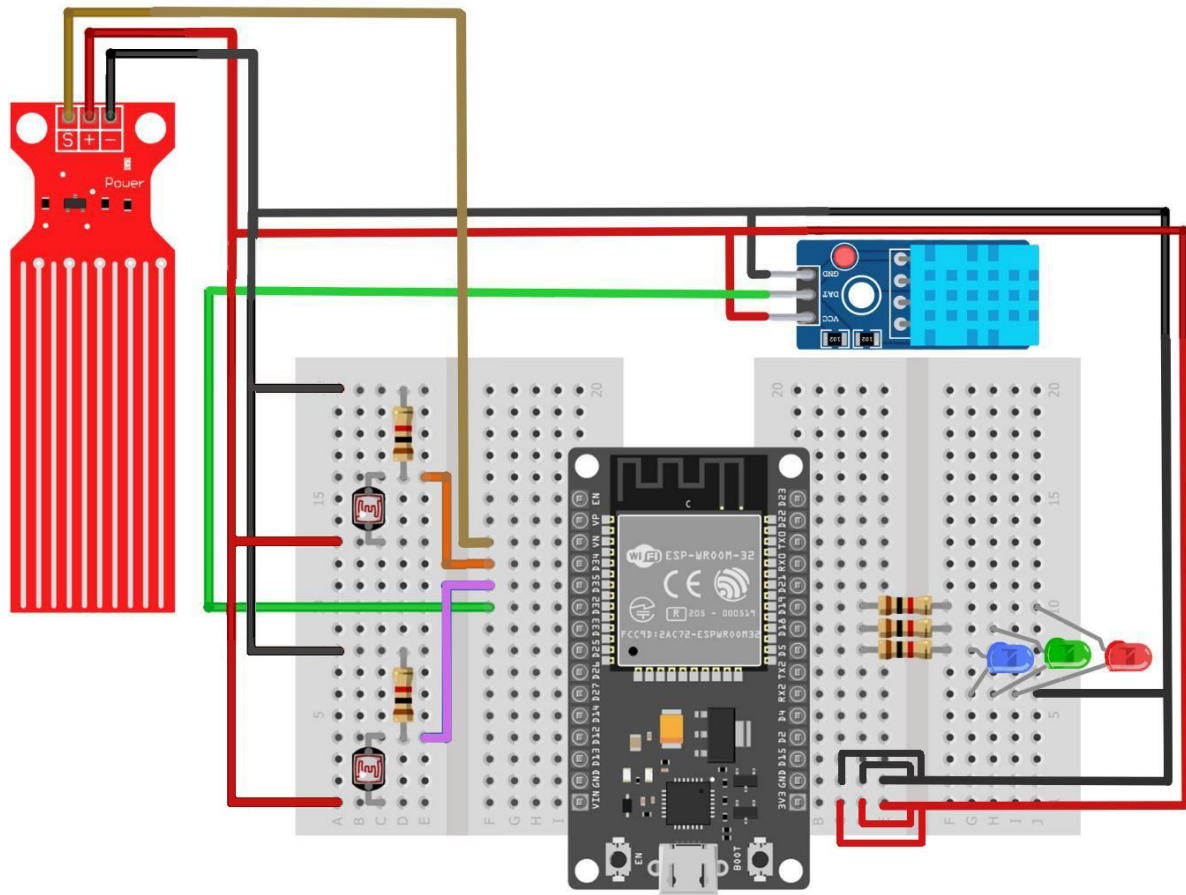
// Bloque de inicialización
void setup() {

    // Inicializamos el monitor serial
    Serial.begin(115200);
}

// Bucle principal del código
void loop() {

    // Tomamos la lectura del pin donde se encuentra la LDR
    int lectura_ldr_1 = analogRead(ldrPin1);
    // Mostramos el valor por el monitor Serial
    Serial.print("Luminosidad 1: ");
    Serial.println(lectura_ldr_1);
}
```

A continuación se debe añadir la segunda LDR con su resistencia. Para obtener el valor de la lectura de este sensor se va a utilizar la salida digital, como en el caso anterior, pero esta se va a conectar al **pin D35**. A continuación se puede ver el montaje con ambas LDR para la parte actual del Demostrador.



Comprobemos que se ha conectado correctamente empleando el siguiente código, que es muy similar al anterior pero mostrando el valor de la medida de ambas fotorresistencias.

```
#define ldrPin1 34 // Definimos el pin donde está conectado la primera LDR
#define ldrPin2 35 // Definimos el pin donde está conectado la segunda LDR

// Bloque de inicialización
void setup() {

    // Inicializamos el monitor serial
    Serial.begin(115200);

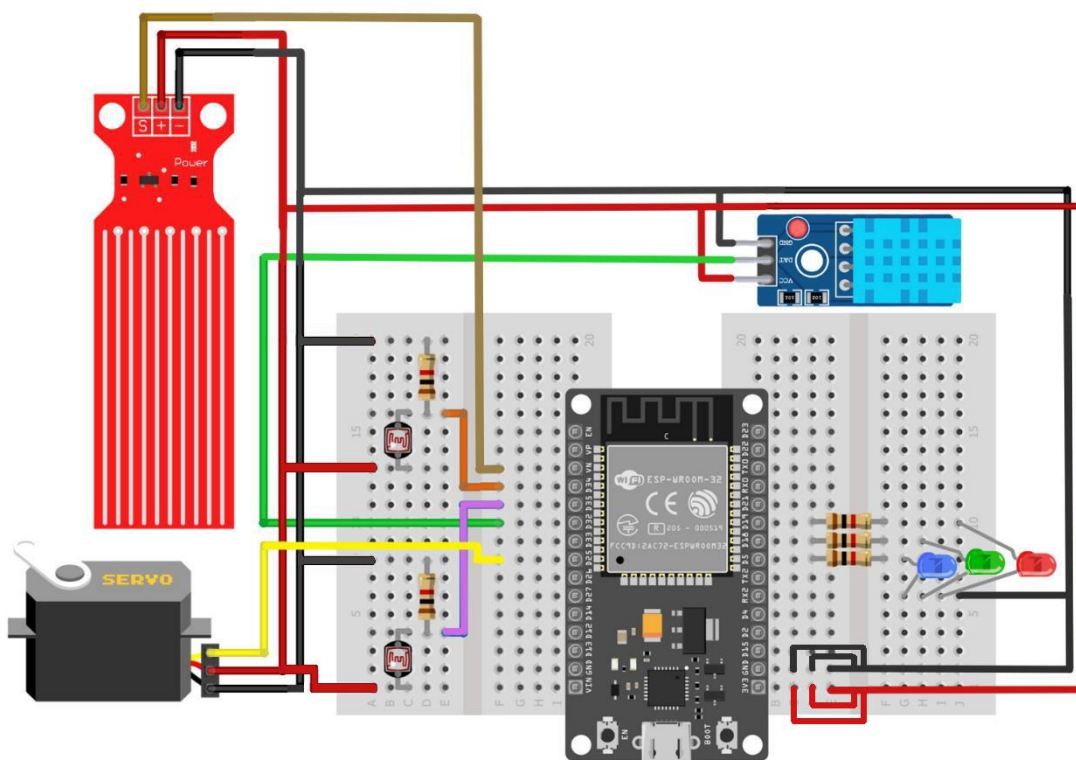
}
```

```
// Bucle principal del código
void loop() {

    // Tomamos las lecturas de los pines donde se encuentran las LDR
    int lectura_ldr_1 = analogRead(ldrPin1);
    int lectura_ldr_2 = analogRead(ldrPin2);
    // Mostramos el primer valor por el monitor Serial
    Serial.print("Luminosidad 1: ");
    Serial.println(lectura_ldr_1);
    // Mostramos el segundo valor por el monitor Serial
    Serial.print("Luminosidad 2: ");
    Serial.println(lectura_ldr_2);

}
```

El siguiente componente que se va a añadir en el montaje es un servomotor. Este se va a conectar a ambas líneas de alimentación (cable rojo a la positiva y cable negro a la negativa) y el cable amarillo (que es la señal de control) se va a conectar al **pin D25**.



Comprobaremos el funcionamiento del servomotor cargando el siguiente código en la placa ESP32 que realiza un barrido moviendo el servo entre las posiciones de 0°, 90° y 180°, en bucle, una y otra vez.

```
#include <ESP32Servo.h> // Incluir la librería necesaria para controlar el servomotor

#define servoPin 25 // Definimos el pin donde está conectado el servomotor

// Creamos un objeto de tipo Servo para controlarlo
Servo myServo;

// Bloque de inicialización
void setup() {

    // Asociamos el servo que creamos al pin donde está conectado
    myServo.attach(servoPin);
    // Colocamos el servo en una posición inicial de 0°
    myServo.write(0);

}

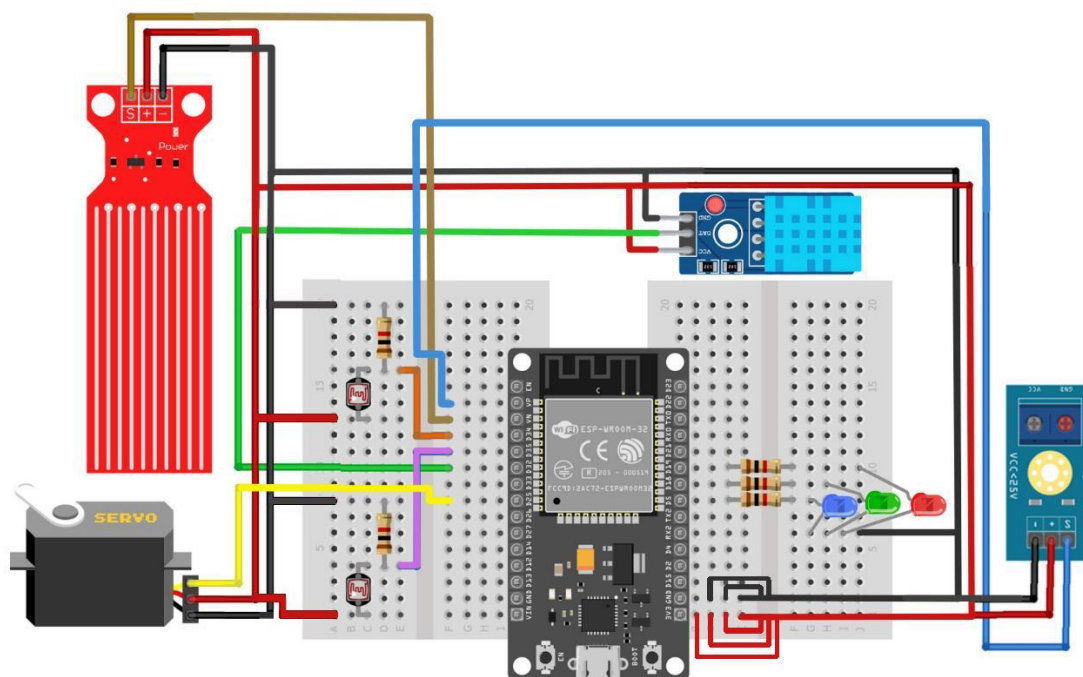
// Bucle principal del código
void loop() {

    myServo.write(0); // Movemos el servomotor a 0°
    delay(1000); // Esperamos 1 segundo = 1000 milisegundos
    myServo.write(90); // Movemos el servomotor a 90°
    delay(1000); // Esperamos 1 segundo
    myServo.write(180); // Movemos el servomotor a 180°
    delay(1000); // Esperamos 1 segundo

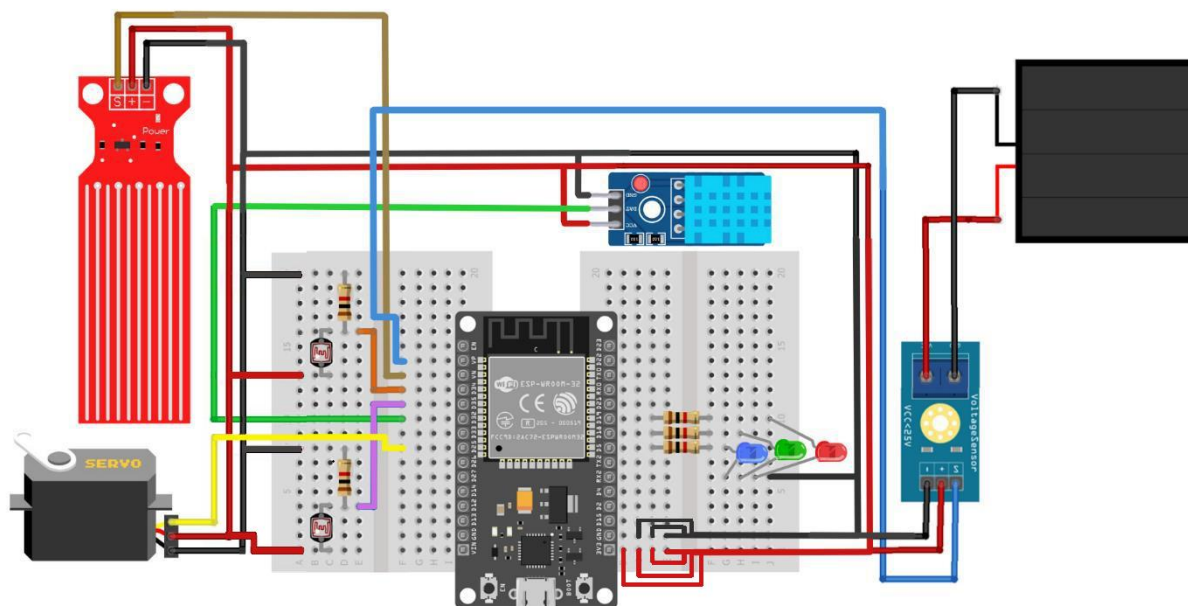
}
```

Nota: Colocar la pestaña en el servo. No es necesario usar tornillos para sujetarla.

El último componente que vamos a conectar al ESP32 será un detector de tensión. Este módulo se debe alimentar a ambas líneas de alimentación (Vcc y GND). La línea de señal se va a conectar al **pin VP**.



El último paso que falta para terminar el montaje electrónico es conectar el panel solar al detector de tensión. A continuación, se ve cómo quedaría el montaje completo.



Vamos a comprobar el funcionamiento del detector solar completo haciendo uso del siguiente código que simplemente lee la señal del módulo detector de tensión y enciende el LED Azul si detecta voltaje.

```
#define ledPin1Cold 5 // Definimos el pin donde está conectado el LED Azul
#define detectorPin 36 // Definimos el pin donde está conectado el Detector de Tensión

// Bloque de inicialización
void setup() {

    // Configuramos el modo de funcionamiento del pin del LED como salida
    pinMode(ledPin1Cold, OUTPUT);

}

// Bucle principal del código
void loop() {

    // Tomamos la lectura del Módulo Detector
    bool tension_detectada = digitalRead(detectorPin);
    // Comprobamos si hay tensión o no
    if(tension_detectada) { // Sí hay tensión

        digitalWrite(ledPin1Cold, HIGH); // Encendemos el LED Azul

    } else { // No hay tensión

        digitalWrite(ledPin1Cold, LOW); // Apagamos el LED Azul

    }

}
```

A continuación, dejamos un código test para comprobar que todos los sensores obtienen lecturas y funcionan correctamente:

```
//include the necessary libraries

#include <ESP32Servo.h> // Libreria para controlar servomotores en el ESP32
#include "DHT.h" // Libreria para leer el sensor de temperatura/humedad DHT

// ----- Pines -----
const int LDR_PIN_EAST = 34; // Pin analogico donde esta conectado el LDR del lado Este
```

```

const int LDR_PIN_WEST = 35; // Pin analogico donde esta conectado el LDR del lado Oeste
const int DHT_PIN = 32; // Pin digital donde esta conectado el sensor DHT11
const int LED_COLD_PIN = 5; // Pin digital del LED azul (indica frio)
const int LED_GOOD_PIN = 18; // Pin digital del LED verde (indica temperatura normal)
const int LED_HEAT_PIN = 19; // Pin digital del LED rojo (indica calor)
const int SOLAR_PIN = 36; // Pin analogico donde esta conectado el panel solar
const int WATER_PIN = 39; // Pin analogico donde esta conectado el sensor de nivel de agua
const int SERVO_PIN = 25; // Pin PWM donde esta conectado el servomotor

// ----- Constantes de comportamiento -----
const int SERVO_TOLERANCE = 20; // Diferencia minima entre LDRs para mover el servo
const float SOLAR_THRESHOLD_UV = 10.0; // Umbral minimo en microvoltios para considerar luz solar valida
const unsigned long LOOP_DELAY = 5000; // Tiempo de espera (ms) entre cada ciclo del loop

#define DHTTYPE DHT11 // Define el modelo de sensor DHT que se esta usando (DHT11)

// ----- Objetos globales -----
DHT dht(DHT_PIN, DHTTYPE); // Crea el objeto "dht" asociado al pin y tipo de sensor definidos
Servo myservo; // Crea el objeto "myservo" para controlar el servomotor

// ----- Estado -----
int valldr1 = 0; // Guarda la ultima lectura del LDR Este
int valldr2 = 0; // Guarda la ultima lectura del LDR Oeste
int averageLdrValue = 0; // Guarda el promedio entre ambos LDR
int servopos = 90; // Posicion actual del servo, arranca centrado en 90 grados
float lastMicrovolts = 0; // Guarda el ultimo valor de microvoltios distinto que se imprimio (para detectar cambios)
int sensorValue; // Guarda la lectura cruda del sensor de agua

void setup() {
  Serial.begin(115200); // Inicia la comunicacion serial a 115200 baudios para ver datos en consola

  analogReadResolution(12); // Configura las lecturas analogicas con resolucion de 12 bits (0-4095)
  analogSetAttenuation(ADC_11db); // Configura la atenuacion del ADC para leer voltajes hasta ~3.3V

  pinMode(4, INPUT); // Configura el pin 4 como entrada (reservado, no usado en este codigo)
  pinMode(LED_COLD_PIN, OUTPUT); // Configura el pin del LED azul como salida
  pinMode(LED_GOOD_PIN, OUTPUT); // Configura el pin del LED verde como salida
  pinMode(LED_HEAT_PIN, OUTPUT); // Configura el pin del LED rojo como salida
  pinMode(LDR_PIN_EAST, INPUT); // Configura el pin del LDR Este como entrada
  pinMode(LDR_PIN_WEST, INPUT); // Configura el pin del LDR Oeste como entrada

  dht.begin(); // Inicializa el sensor DHT para empezar a leer
  myservo.attach(SERVO_PIN); // Asocia el objeto servo al pin fisico definido
  myservo.write(servopos); // Mueve el servo a su posicion inicial (90 grados)

  delay(1000); // Espera 1 segundo para que todos los sensores se estabilicen
}

void loop() {
  readSolarPanel(); // Lee el panel solar e imprime su resultado
  readTemperatureHumidity(); // Lee temperatura/humedad, actualiza LEDs e imprime el resultado
  readWaterLevel(); // Lee el sensor de agua e imprime el resultado
  readLdrAndMoveServo(); // Lee los LDR, mueve el servo e imprime el resultado

  delay(LOOP_DELAY); // Espera antes de iniciar el siguiente ciclo de lectura
}

```

```
// =====
// Lectura del panel solar
// =====
void readSolarPanel() {
  Serial.println("----- Measured Solar (V) ----- "); // Encabezado de
  esta seccion en consola

  int solarVoltagedV = analogRead(SOLAR_PIN); // Lee el valor crudo del ADC (0-4095) en el pin del panel solar
  int solarVoltage = solarVoltagedV * 10; // Escala el valor crudo multiplicando por 10 (factor de calibracion)

  Serial.print("Voltage of Solar Panel: "); // Imprime etiqueta del voltaje escalado
  Serial.println(solarVoltage); // Imprime el valor del voltaje escalado

  float voltage = (solarVoltage / 4095.0) * 3.3; // Convierte el valor escalado a voltios reales (0-3.3V)

  Serial.print("Measured voltage (V): "); // Imprime etiqueta del voltaje medido
  Serial.println(voltage); // Imprime el voltaje medido en voltios

  float microvoltsSolar = voltage * 1000000.0; // Convierte el voltaje de voltios a microvoltios

  Serial.print("Input voltage (uV): "); // Imprime etiqueta del voltaje en microvoltios
  Serial.println(microvoltsSolar); // Imprime el voltaje en microvoltios

  if (microvoltsSolar < SOLAR_THRESHOLD_UV) { // Si la lectura esta por debajo del umbral definido...
    microvoltsSolar = 0; // ...se considera ruido y se fuerza a cero
  }

  if (microvoltsSolar != lastMicrovolts) { // Si el valor cambio respecto a la ultima lectura registrada...
    Serial.print("Send voltage: "); // ...imprime etiqueta indicando el nuevo valor
    Serial.print(microvoltsSolar, 2); // ...imprime el nuevo valor con 2 decimales
    Serial.println(" uV"); // ...imprime la unidad (microvoltios)

    lastMicrovolts = microvoltsSolar; // Actualiza el ultimo valor registrado para la proxima comparacion
  }
}

// =====
// Lectura de temperatura y humedad + control de LEDs
// =====
void readTemperatureHumidity() {
  Serial.println("----- Measured Temperature / Humidity ----- ");

  float h = dht.readHumidity(); // Lee la humedad relativa del sensor DHT (%)
  float t = dht.readTemperature(); // Lee la temperatura del sensor DHT (grados Celsius)

  if (isnan(h) || isnan(t)) { // Si alguna de las dos lecturas falla (valor invalido)...
    Serial.println("Failed to read from DHT sensor!"); // ...imprime mensaje de error
    return; // ...y sale de la funcion sin continuar
  }

  Serial.print("Temperature: "); // Imprime etiqueta de temperatura
  Serial.print(t); // Imprime el valor de temperatura
  Serial.print(" C, Humidity: "); // Imprime unidad de temperatura y etiqueta de humedad
```

```

Serial.print(h);          // Imprime el valor de humedad
Serial.print(" %");       // Imprime unidad de humedad
Serial.println();         // Salto de línea final

updateTemperatureLeds(t); // Actualiza el estado de los LEDs segun la temperatura leida
}

// Enciende el LED correspondiente segun el rango de temperatura
void updateTemperatureLeds(float t) {
  if (t > 30) {            // Si la temperatura es mayor a 30 grados (calor)...
    digitalWrite(LED_HEAT_PIN, HIGH); // ...enciende el LED rojo
    digitalWrite(LED_GOOD_PIN, LOW);  // ...apaga el LED verde
    digitalWrite(LED_COLD_PIN, LOW);  // ...apaga el LED azul
    Serial.println("Heat (rojo) is HIGH"); // Informa por consola que el LED de calor esta encendido
  } else if (t < 20) {     // Si la temperatura es menor a 20 grados (frio)...
    digitalWrite(LED_HEAT_PIN, LOW);  // ...apaga el LED rojo
    digitalWrite(LED_GOOD_PIN, LOW);  // ...apaga el LED verde
    digitalWrite(LED_COLD_PIN, HIGH); // ...enciende el LED azul
    Serial.println("Cold (azul) is HIGH"); // Informa por consola que el LED de frio esta encendido
  } else {                // Si la temperatura esta entre 20 y 30 grados (rango normal)...
    digitalWrite(LED_HEAT_PIN, LOW);  // ...apaga el LED rojo
    digitalWrite(LED_GOOD_PIN, HIGH); // ...enciende el LED verde
    digitalWrite(LED_COLD_PIN, LOW);  // ...apaga el LED azul
    Serial.println("Good (verde) is HIGH"); // Informa por consola que el LED de rango normal esta encendido
  }
}

// =====
// Lectura del sensor de nivel de agua
// =====
void readWaterLevel() {
  Serial.println("----- Measured Water Level ----- ");

  sensorValue = analogRead(WATER_PIN); // Lee el valor crudo del ADC en el pin del sensor de agua

  if (sensorValue >= 2000) { // Si la lectura llega o supera el limite esperado (2000)...
    sensorValue = 1980;    // ...la recorta a 1980 para evitar valores fuera de rango
  }

  float waterlevel = (sensorValue * 100) / 2000; // Convierte la lectura cruda a porcentaje (0-100%)

  Serial.print("Water level: "); // Imprime etiqueta de nivel de agua
  Serial.print(waterlevel);      // Imprime el valor de nivel de agua
  Serial.println(" %");          // Imprime unidad (porcentaje)
}

// =====
// Lectura de LDRs y movimiento del servo
// =====
// Este codigo esta preparado para leer los datos de los LDR de modo que a mayor cantidad de luz,
// mayor sera la respuesta analogica en consola. Por lo que el hardware, en el divisor de tension, en el palo del LDR
// debe estar conectado a VCC y el palo de la resistencia a GND.

void readLdrAndMoveServo() {
  Serial.println("----- Measured LDR and Servo ----- ");

  valldr1 = analogRead(LDR_PIN_EAST); // Lee el valor analogico del LDR ubicado al Este
  valldr2 = analogRead(LDR_PIN_WEST); // Lee el valor analogico del LDR ubicado al Oeste

```

```
averageLdrValue = (valldr1 + valldr2) / 2; // Calcula el promedio entre ambas lecturas de luz

Serial.print("LDR 1: "); // Imprime etiqueta del LDR Este
Serial.print(valldr1);    // Imprime el valor del LDR Este
Serial.print(" | LDR 2: "); // Imprime separador y etiqueta del LDR Oeste
Serial.println(valldr2);  // Imprime el valor del LDR Oeste

Serial.print("Average LDR: "); // Imprime etiqueta del promedio
Serial.println(averageLdrValue); // Imprime el valor promedio de ambos LDR

if ((abs(valldr1 - valldr2) <= SERVO_TOLERANCE) || (abs(valldr2 - valldr1) <= SERVO_TOLERANCE)) { // Si la diferencia
de luz esta dentro de la tolerancia...
    // No se mueve el servo, se mantiene en su posicion actual
} else { // Si la diferencia de luz supera la tolerancia...
    if (valldr1 > valldr2) { // ...y hay mas luz en el Este que en el Oeste...
        servopos = servopos + 10; // ...aumenta el angulo del servo 10 grados
    }

    if (valldr1 < valldr2) { // ...y hay mas luz en el Oeste que en el Este...
        servopos = servopos - 10; // ...disminuye el angulo del servo 10 grados
    }
}

if (servopos > 180) { // Si el angulo calculado supera el maximo permitido...
    servopos = 180; // ...lo limita a 180 grados
}

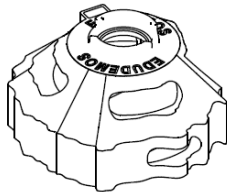
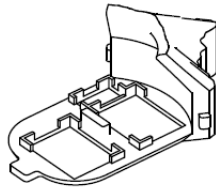
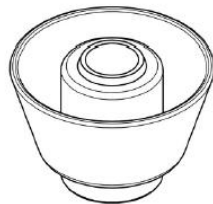
if (servopos < 0) { // Si el angulo calculado es menor al minimo permitido...
    servopos = 0; // ...lo limita a 0 grados
}

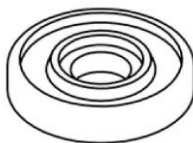
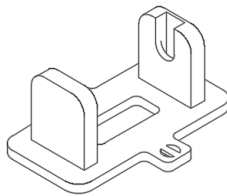
myservo.write(servopos); // Aplica el angulo final calculado al servomotor

Serial.print("Servo angle: "); // Imprime etiqueta del angulo del servo
Serial.println(servopos);    // Imprime el angulo final aplicado al servo
}
```

1. 2. 3. Montaje Mecánico del Demostrador de Agua + Solar

Los elementos que componen este demostrador son los siguientes:

Elemento	Cantidad	Descripción	Imagen
Base	1	Es la parte superior que asegura los componentes móviles, como las palas y el eje del motor. Proporciona estabilidad y permite que el sistema funcione suavemente, facilitando la correcta transferencia de energía desde las palas al generador.	
Cajón	1	Es la base que conecta las diferentes secciones del dispositivo, proporcionando soporte estructural y asegurando una correcta alineación entre el motor, las palas y otros componentes esenciales para su funcionamiento.	
Contenedor de agua	1	Esta parte es capaz de almacenar el agua que cae en él.	
Hoja	8	Hoja que permite que los líquidos entren en el contenedor de agua	

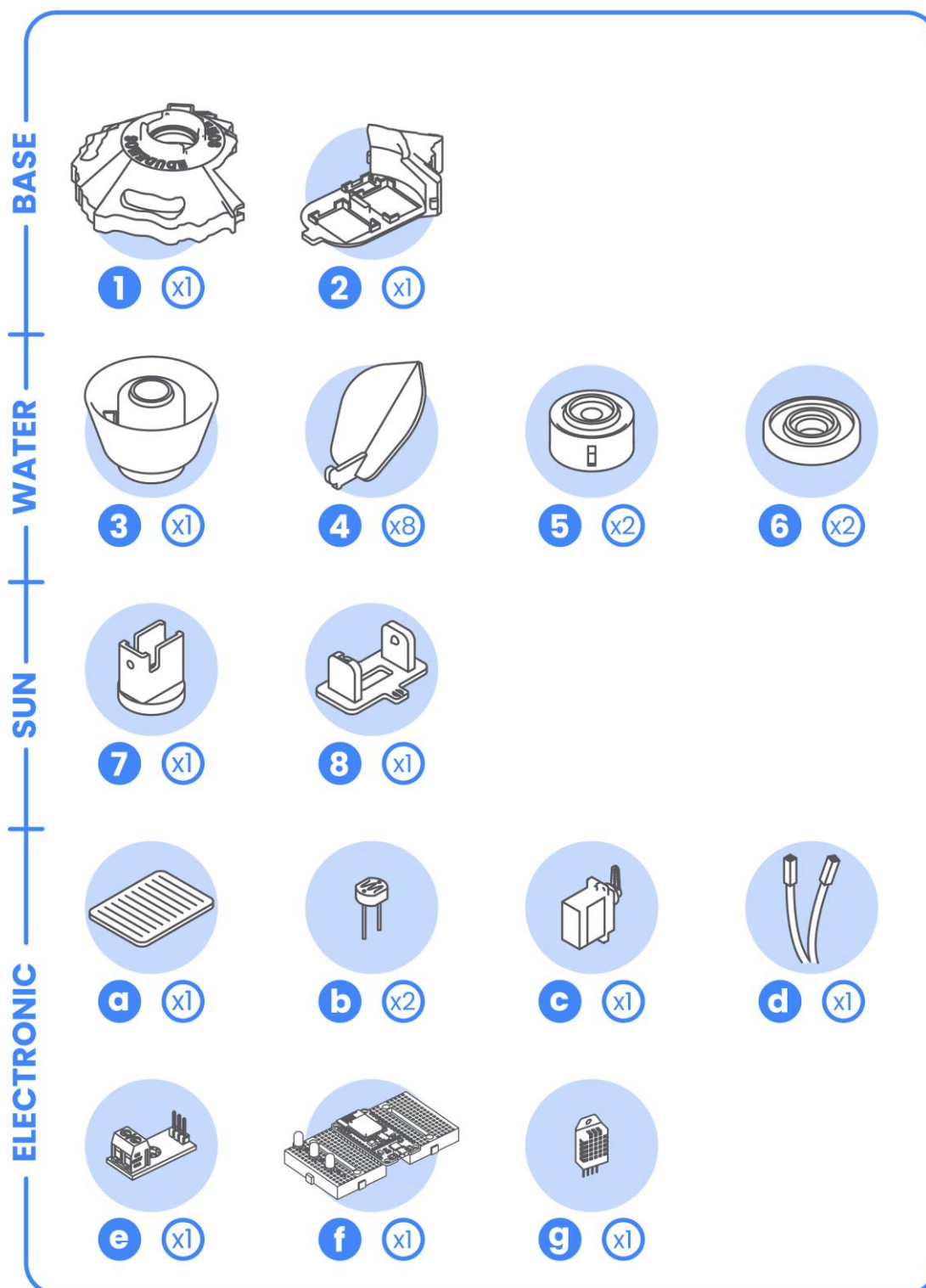
Disco Pequeño	3	Es un componente estático que no se mueve. Su función principal es actuar como parte del soporte estructural del dispositivo, ayudando a mantener la correcta alineación y estabilidad de los elementos.	
Disco Estático	2	Disco que se mantiene fija para tapar y mantener la estabilidad de la estructura.	
Placa Sol	1	Parte que permite el movimiento del panel solar para su correcta orientación	
Disco Solar	1	Disco que permite conectar el girador solar con un servomotor instalado y juntarlo con el resto de la estructura.	

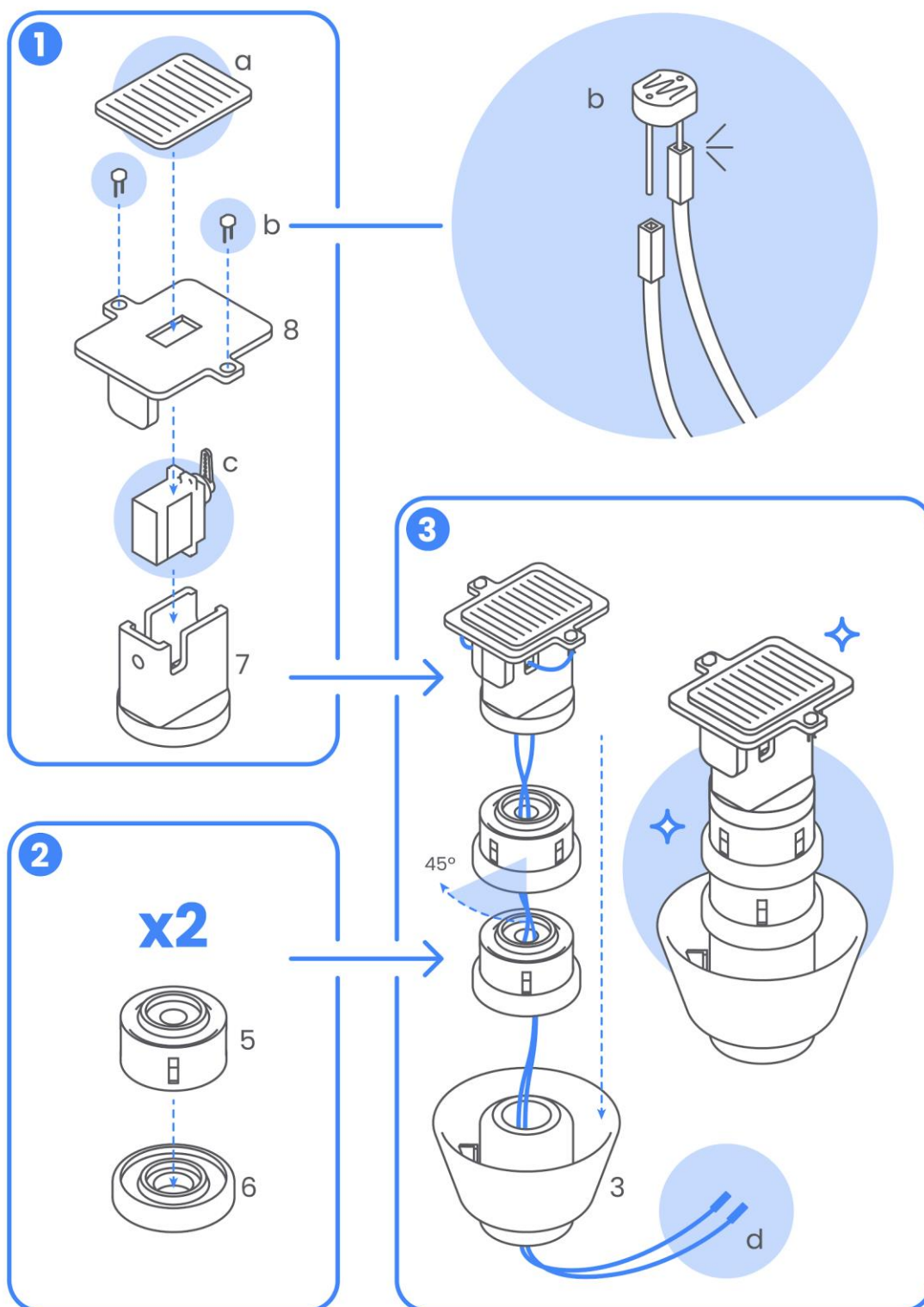
A continuación se muestra cómo se realiza el montaje del demostrador:

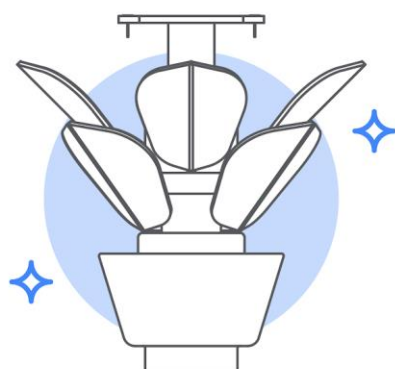
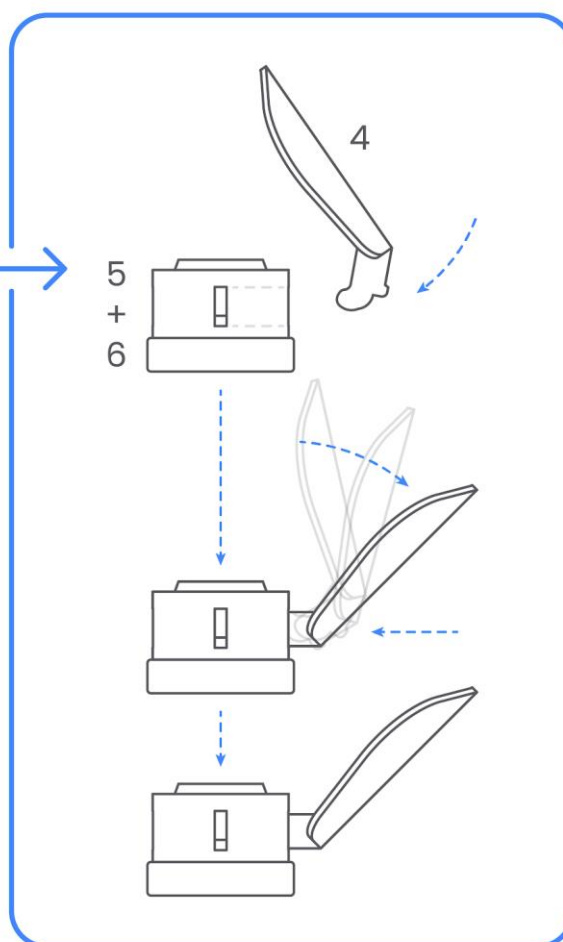
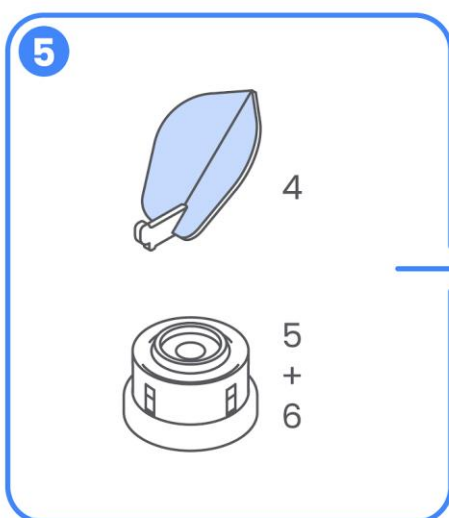
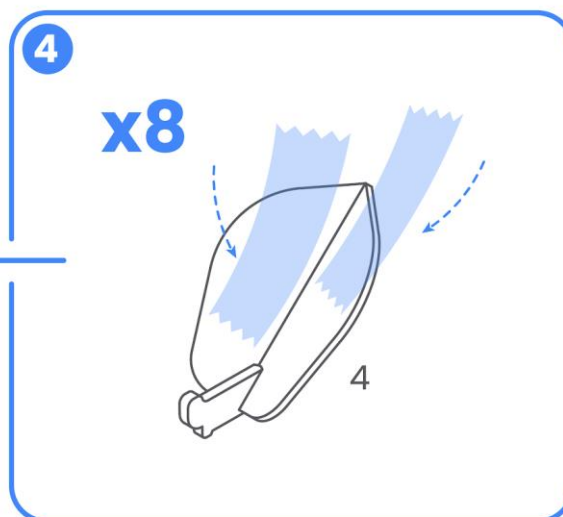
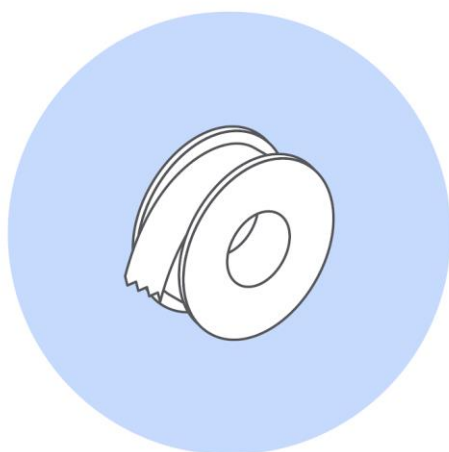
Nota: La cinta de aluminio se utiliza en la superficie de las hojas para optimizar la recolección de agua. La cinta de doble cara se utiliza en la parte trasera del panel solar para pegarlo en la pieza del "placa sol".

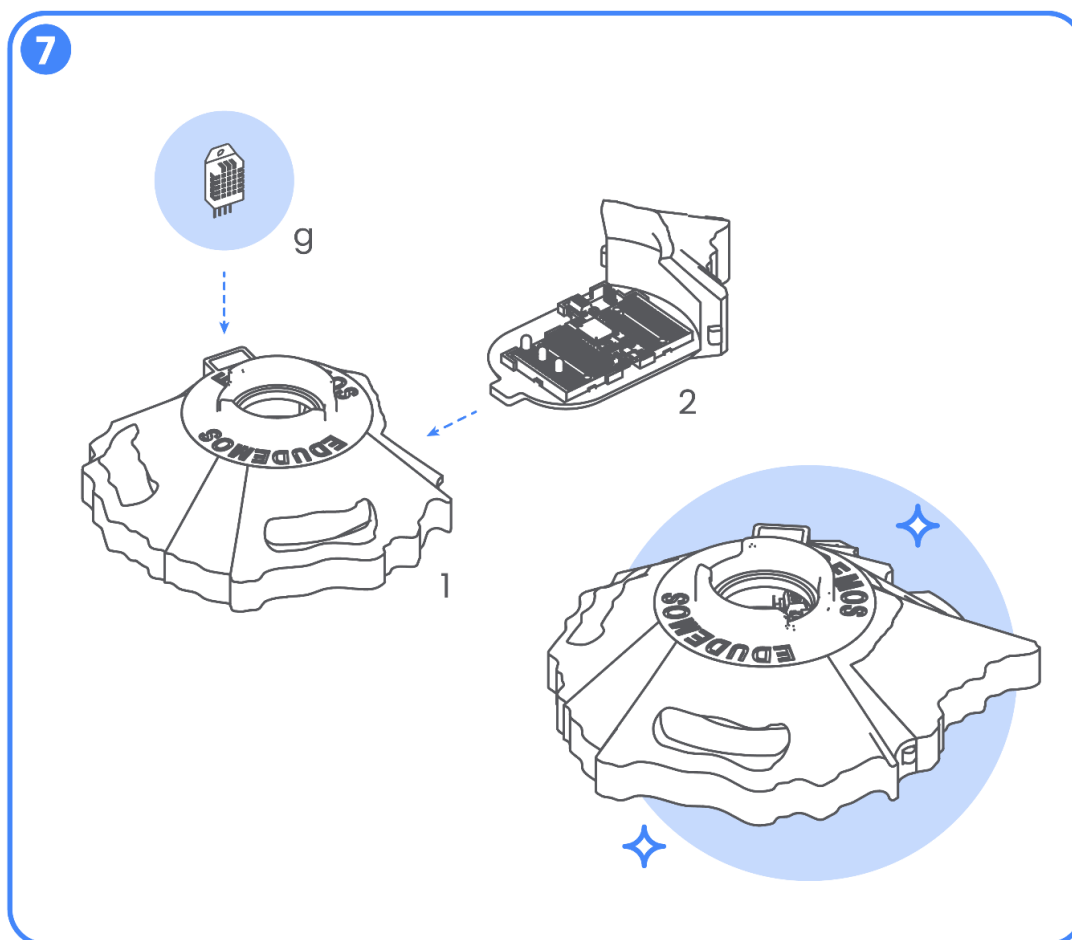
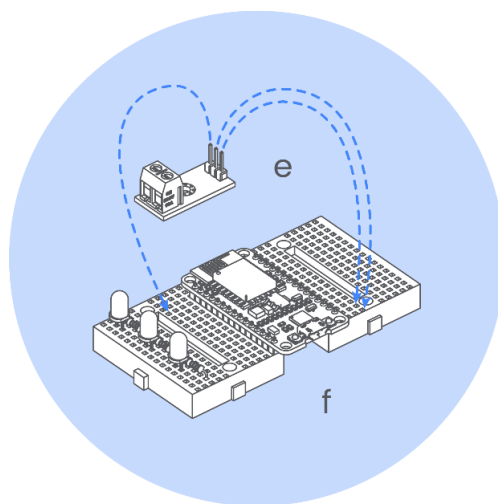
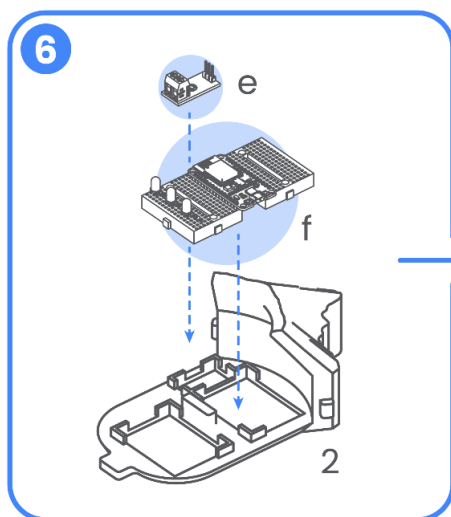
El sensor de nivel de agua va colocado en unas ranuras dentro de la pieza "contenedor de agua".

Al colocar los componentes electrónicos en el montaje mecánico, es necesario usar cables adicionales para alargar las conexiones de las LDR y de la placa solar.

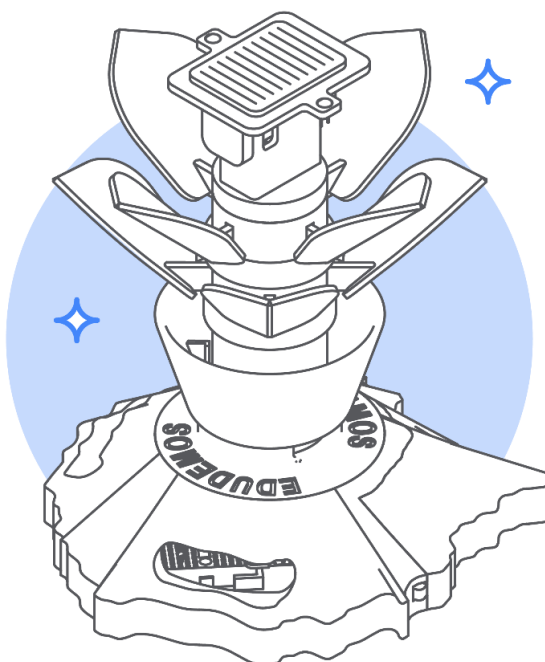
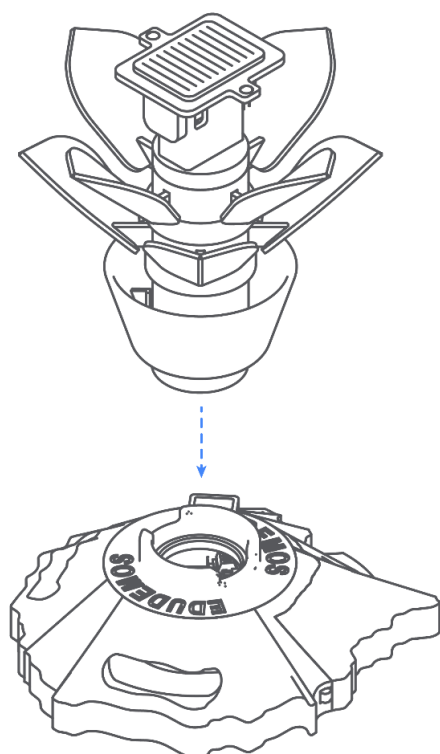








8



Co-funded by
the European Union

1. 3. Montaje del Demostrador de viento

1. 3. 1. Montaje electrónico

Comenzaremos realizando el montaje del Demostrador de viento. Para realizar este montaje completo necesitarás el siguiente material:



1 x Motor de Corriente Continua de 5V

Empleado para generar voltaje con la fuerza del viento.



1 x Condensador de 100nF / 1uF

Utilizados para eliminar ruidos indeseados generados por el Motor de CC.



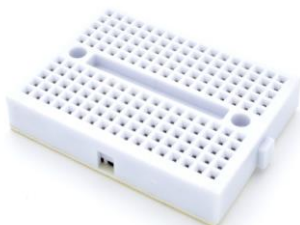
2 x Resistencias de 1 kΩ

Utilizados en el divisor de tensión que permite tomar la lectura de tensión generada por el Motor de CC.



6 x Cable dupont (Macho-Macho) 10cm

Necesarios para realizar las conexiones entre los diferentes componentes en la protoboard.



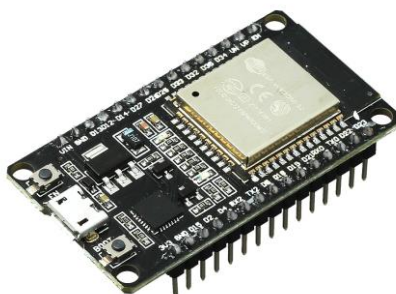
2 x Mini Breadboard

Componente necesario para conectar la placa microcontroladora con los diferentes componentes



Soldador de estaño

sólo si el motor no tiene cables incluidos



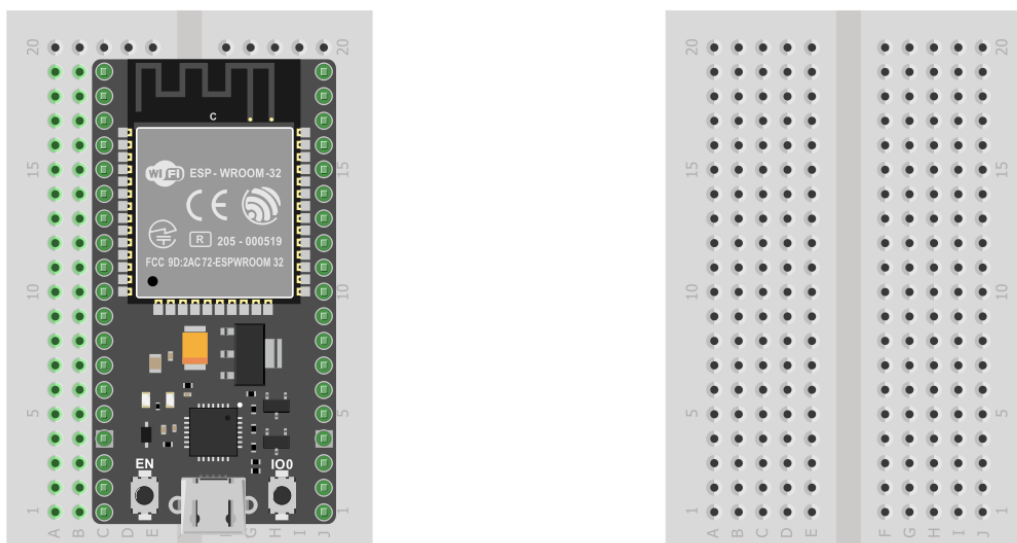
1 x ESP32 DEVKIT V1 (30 pins)

Encargado de ejecutar la programación que controla el funcionamiento de la electrónica.

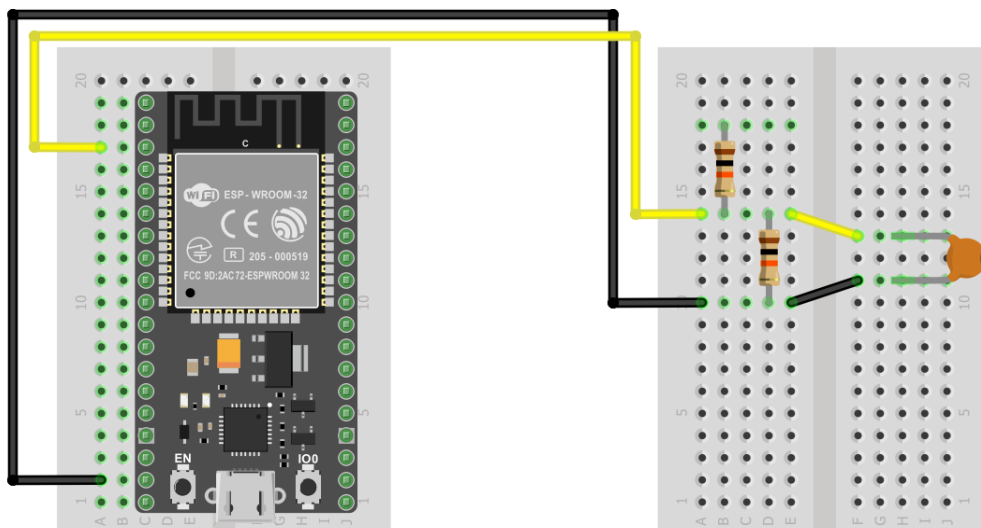
El objetivo del montaje será obtener la medida de la cantidad de voltaje generado por el motor, que irá unido a la estructura mecánica que girará por efecto de la fuerza del viento. Para poder realizar la medición con el ESP32 de la tensión obtenida por el movimiento de las aspas del Demostrador, debemos montar un partidor de tensión que

reduzca el nivel máximo de tensión a obtener (5V) a uno que sea apto para el ESP32 (que trabaja a 3'3V). Con esto podremos obtener cuánto voltaje se está generando por efecto del viento.

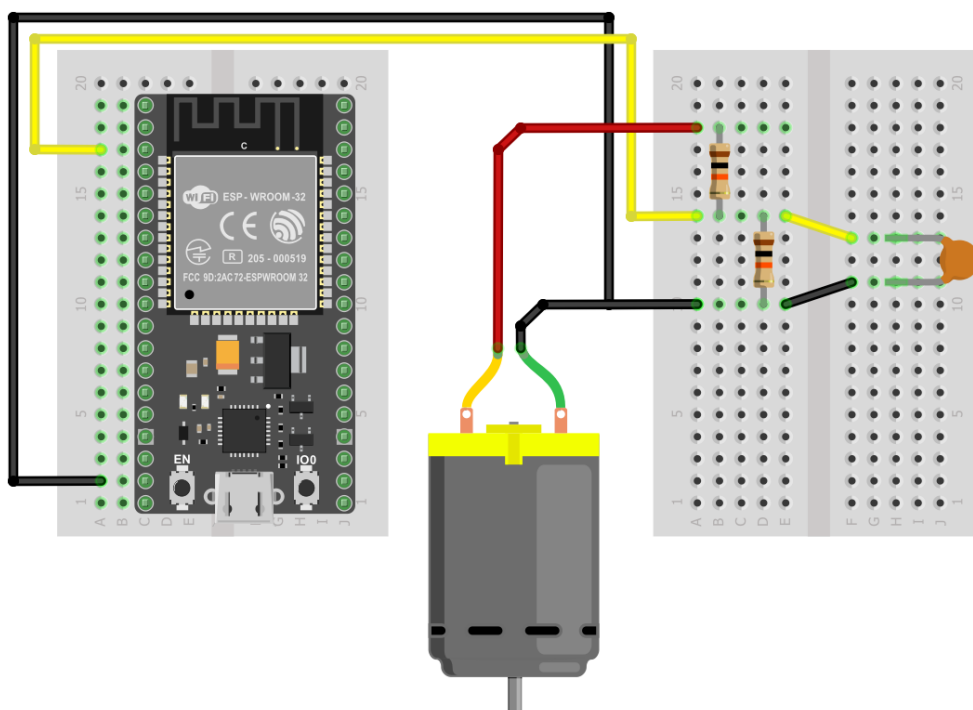
Para este nuevo montaje, será necesario crear un circuito desde el principio. Comenzaremos colocando el ESP32 en dos mini protoboards. A continuación, se muestra el montaje correspondiente.



A continuación, se procederá a montar un divisor de tensión. El punto intermedio de este divisor se conectará al pin **D39 (VN)**. Además, es necesario conectar un condensador entre el terminal de este pin y GND, que servirá de filtro para las variaciones indeseadas en la medida, tal y como se muestra en la imagen siguiente.



El último paso en el montaje electrónico de este demostrador es conectar el motor al conjunto anterior, de manera que uno de sus terminales se une a GND y el otro terminal a la entrada del divisor de tensión montado antes. A continuación, se presenta este montaje terminado.

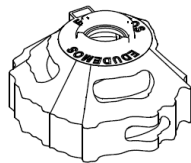
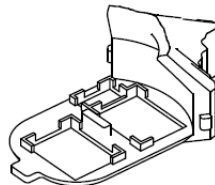


Vamos a comprobar que el montaje de este Demostrador funciona correctamente haciendo uso del siguiente fragmento de código en el que solamente leemos la salida del divisor de tensión que está unida al ESP32. Girando el rotor del motor, veremos como la medición va dando diferentes valores a través del Monitor Serial.

```
// Bloque de inicialización
void setup() {
    // Inicializamos el Monitor Serial
    Serial.begin(115200);
}
// Bucle principal del código
void loop() {
    // Tomamos la medida del pin que va a la salida del partidor de tensión
    int medicion = analogRead(39);
    // Mostramos la medida por el monitor serial
    Serial.println(medicion);
}
```

1.3.2. Montaje Mecánico

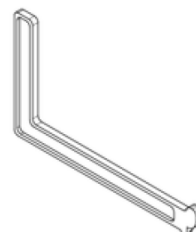
Los elementos que componen este demostrador son los siguientes:

Elemento	Cantidad	Descripción	Imagen
Base	1	Es la parte superior que asegura los componentes móviles, como las palas y el eje del motor. Proporciona estabilidad y permite que el sistema funcione suavemente, facilitando la correcta transferencia de energía desde las palas al generador.	
Cajón	1	Es la base que conecta las diferentes secciones del dispositivo, proporcionando soporte estructural y asegurando una correcta alineación entre el motor, las palas y otros componentes esenciales para su funcionamiento.	
Soporte Motor	1	Esta pieza se encargará de sostener y brindar apoyo al motor, cuyo eje estará conectado al disco de rodamiento. Este diseño permitirá la conexión de los pines de salida, que emitirán señales indicando la presencia de movimiento.	

Hoja Grande en L

6

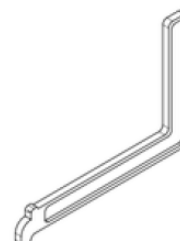
Es una hoja fija que no se mueve. Está diseñada para formar parte de la estructura estática del demostrador, contribuyendo a la estabilidad y el equilibrio del sistema en su conjunto.



Hoja Pequeña en L

6

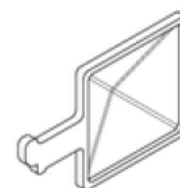
Es una hoja fija que no se mueve, de menor tamaño que la pieza anterior. Está diseñada para formar parte de la estructura estática del demostrador, contribuyendo a la estabilidad y el equilibrio del sistema en su conjunto.



Pala Cuadrada

8

Es una de las palas móviles clave que gira cuando es impulsada por el viento. Su diseño cuadrado permite capturar eficientemente la energía cinética del viento y convertirla en movimiento rotatorio.



Disco grande

2

Es un componente clave que actúa como soporte estructural para las palas móviles. Este disco está diseñado para girar cuando las palas capturan el viento, transfiriendo el movimiento rotatorio hacia el eje central del sistema.



Disco Pequeño

3

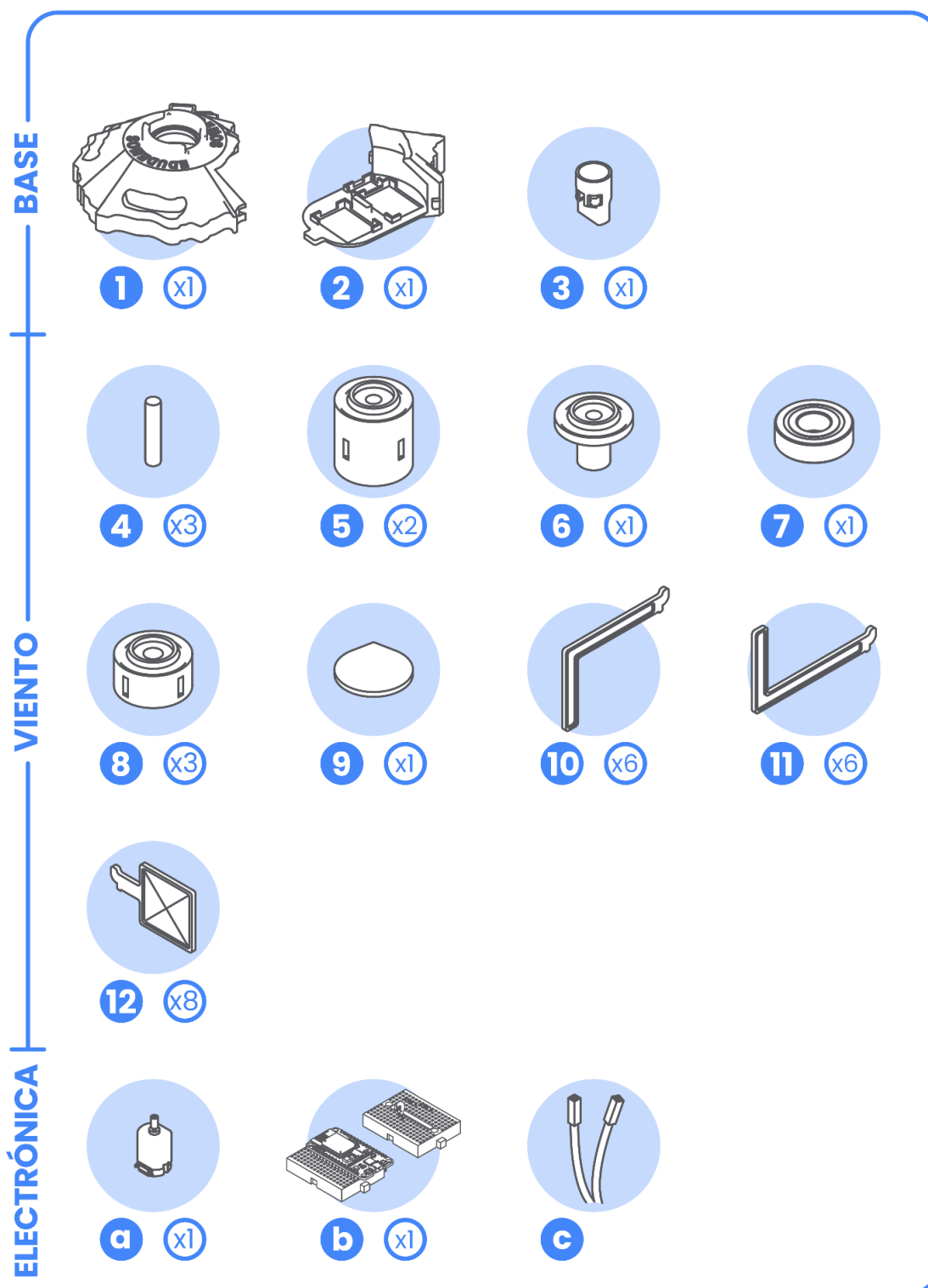
Es un componente estático que no se mueve. Su función principal es actuar como parte del soporte estructural del dispositivo, ayudando a mantener la correcta alineación y estabilidad.

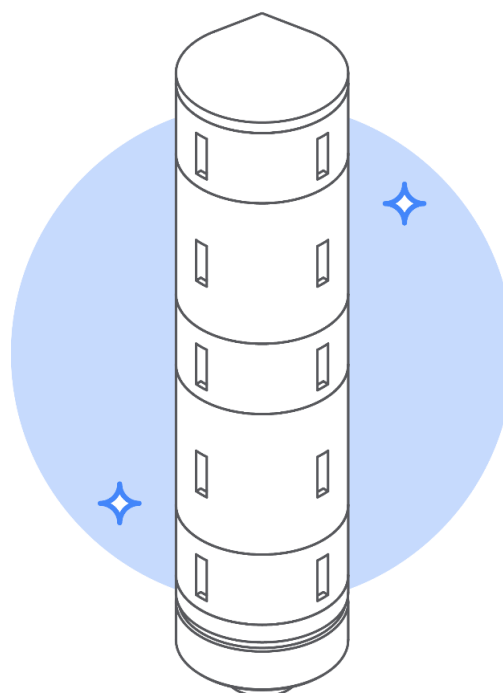
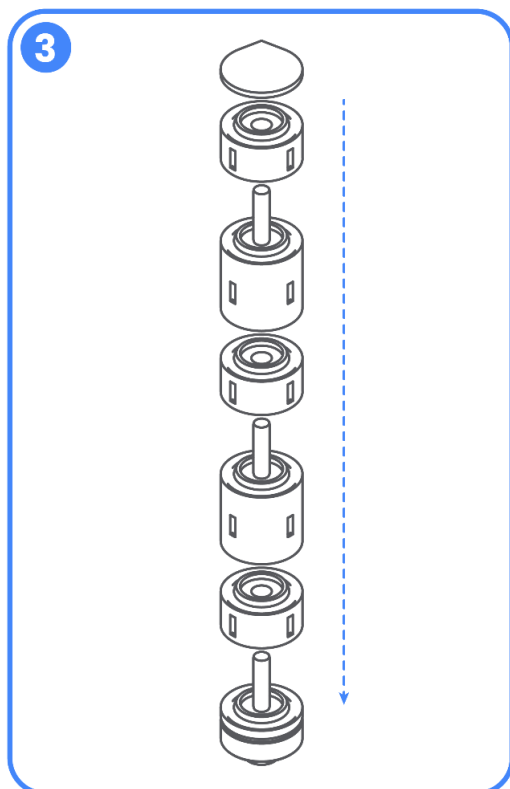
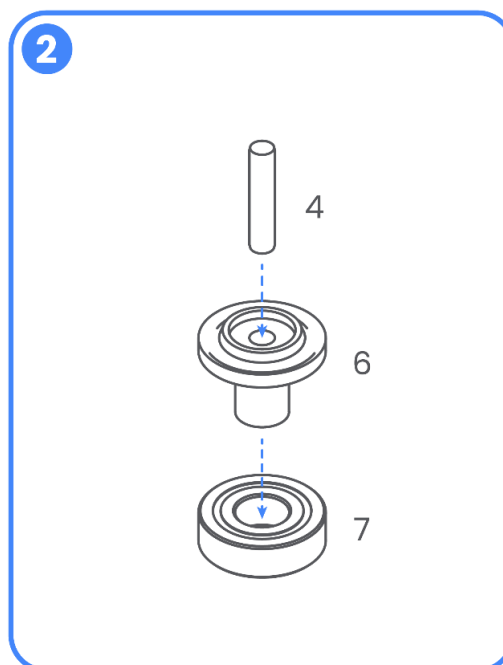
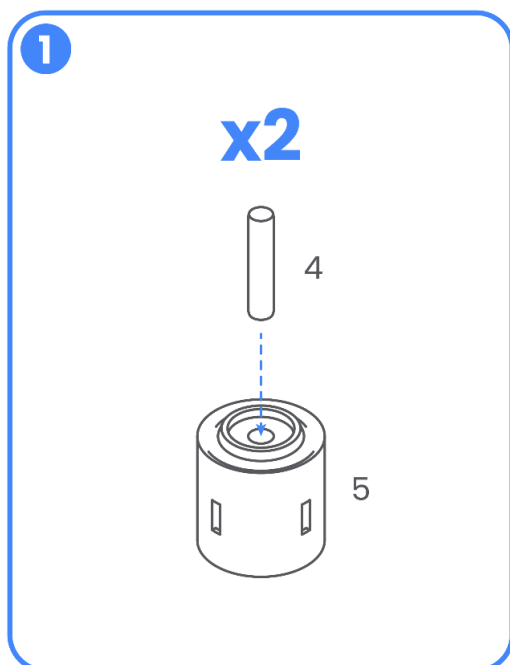


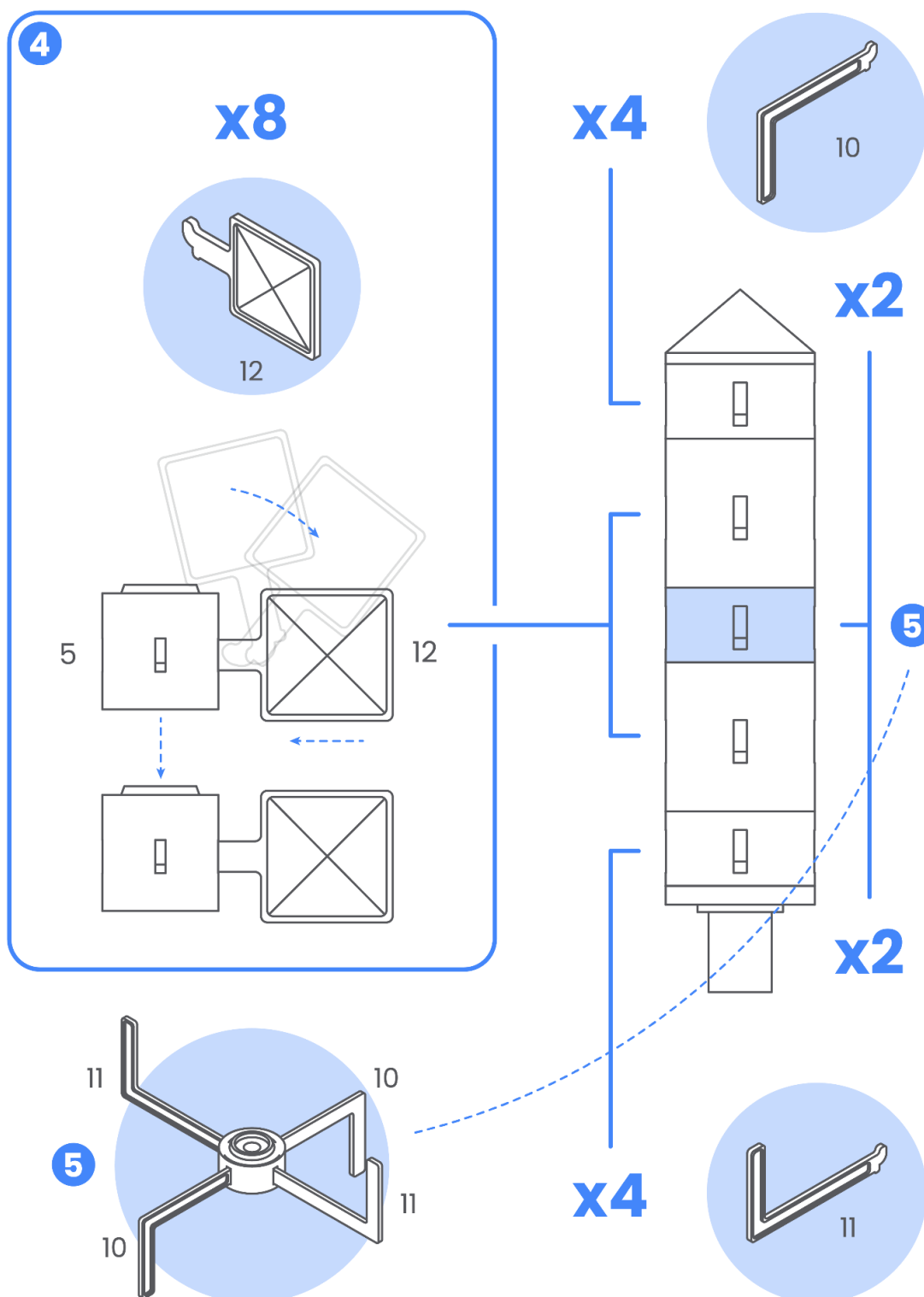
Disco rodamiento	1	de los elementos móviles, como las hojas y los discos grandes.	
		Es un componente crucial que se utiliza para reducir la fricción entre las partes móviles del sistema. Este disco actúa como un soporte rotatorio que permite que los componentes, como el Disco Grande y las palas, giren de manera suave y eficiente alrededor del eje central.	
		Es un componente estructural que se coloca en la parte superior del conjunto, asegurando que todos los elementos internos estén protegidos y correctamente alineados.	
		Es un componente mecánico que permite que los elementos giratorios, como los discos y las palas, se muevan con mínima fricción.	
Tapa	1		
Rodamiento	1		
Espigas Madera	3	Es un componente estructural utilizado para conectar y asegurar las diferentes piezas de madera del dispositivo.	

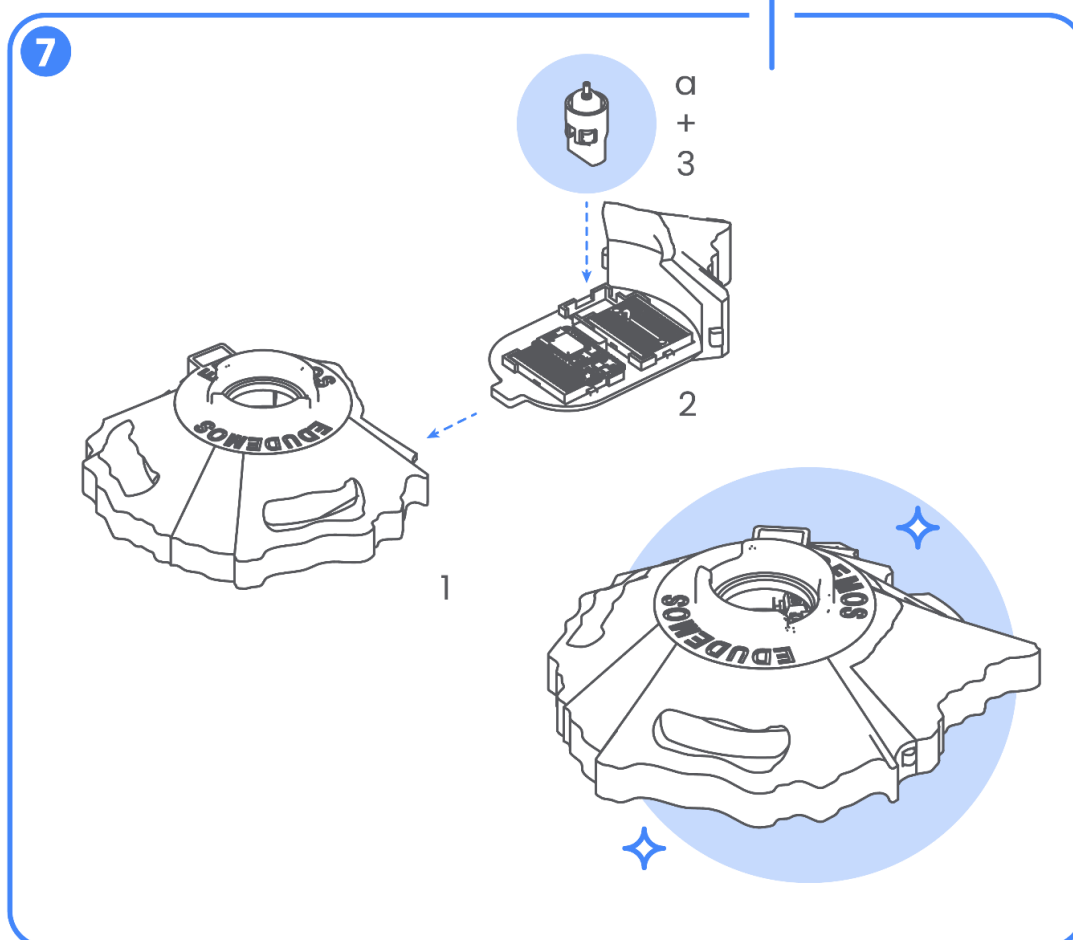
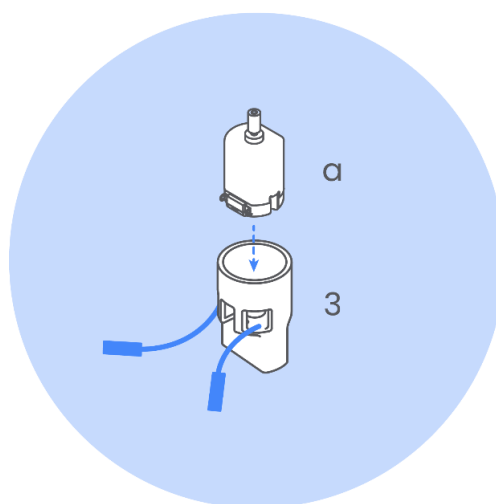
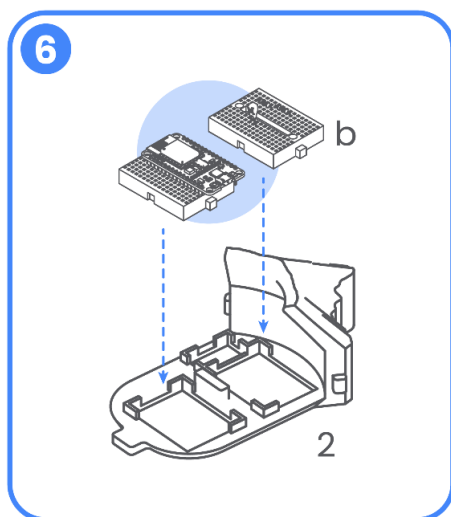
A continuación se muestra cómo se realiza el montaje del demostrador:

Nota: Es necesario insertar el engranaje de 10 dientes en el eje del motor DC antes de comenzar con el montaje del demostrador.

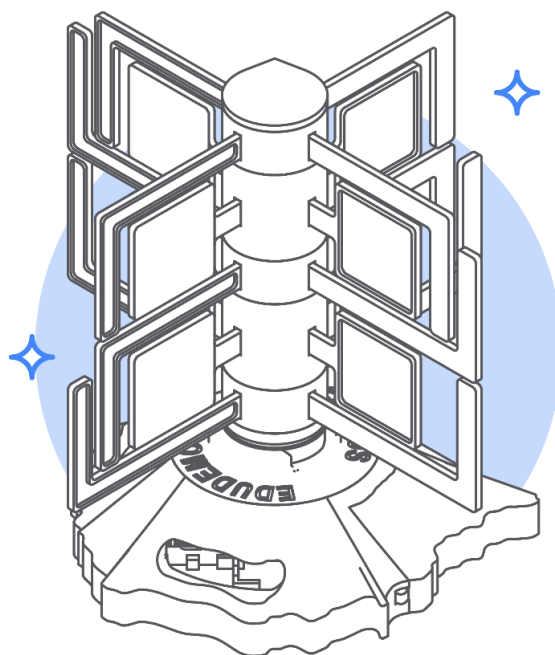
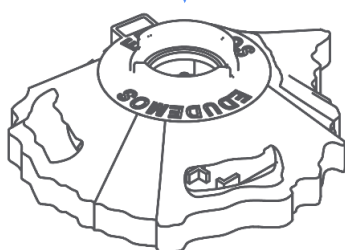
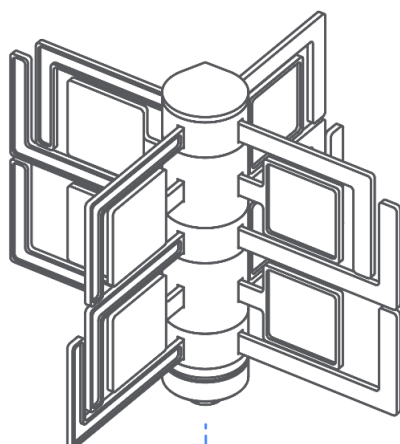








8



Co-funded by
the European Union